

Open Agent Harness Protocol

核心协议 PDF 版。内容来自 docs/harness-protocol/00 到 09 协议文档；不包含独立 RFC 目录。

Generated: 2026-06-02
Project: Open Agent Harness

目录

文件	标题
00-harness-governance-protocol.md	Harness 治理协议总纲
01-agent-model-and-authoring.md	Agent 模型与编写规范
02-model-runtime-protocol.md	Agent Protocol DSL：模型与 Runtime 交互协议
03-action-executor-contract.md	Action 与 Executor 契约
04-routing-and-delegation-policy.md	路由与委托策略
05-handoff-protocol.md	Handoff 协议
06-state-event-projection-model.md	状态、事件、Projection 与 Trace 模型
07-context-memory-visibility-policy.md	上下文、Memory 与 Visibility 策略
08-workflow-profile-action-graph-assets.md	Workflow Profile 与 Action Graph 资产协议
09-ui-console-and-agent-management.md	UI 控制台与 Agent 管理协议

Harness 治理协议总纲

本文档是 Open Agent Harness 的治理协议总纲。它按顺序说明协议目标、设计原则、协议架构、控制流、决策边界、Workflow/UI 定位、共享字段、状态词、对象定义、待讨论问题和文档地图。

01 到 09 是本总纲的分章详细协议，分别展开 Agent 定义与 Skill 来源导入、模型-Runtime 交互、Action/Executor、Routing/Delegation、Handoff、状态、上下文、Workflow Profile 和 UI 管理。总纲聚焦稳定抽象和关系边界。

协议目标

Harness 的目标是建立一个可治理的 agent operating environment。

它面向多个模型会话、工具、工作流、人工决策和长期记忆共同参与任务的场景，让系统保持可控、可审计、可恢复。

设计原则

1. Runtime 控制状态变更

所有会改变系统状态的请求都必须由 Runtime 接受、校验、执行和记录。

Runtime 接受结构化 Command 或 Action，并据此推进 run、修改任务状态、批准审查或替换概念。

2. Agent 会话之间不直接通信

Agent 会话之间的通信和协作由 Runtime 控制协调。

Runtime 通过 Action、Assignment、Event、Projection 和 Artifact 建立协作关系、记录协作过程、推进协作状态。

Agent 模板可以声明默认编排策略，但编排仍由 Runtime 评估、创建和执行。Agent 会话不会因为编排策略而直接调用其他 Agent 会话。

3. 意图采用声明式

模型声明想做什么，而不是直接调用具体工具。Runtime 判断是否允许，再决定如何执行、如何记录结果。

Harness 通过一套 DSL 供模型声明 semantic action graph。Runtime 负责解析、校验、执行、存储、投影和恢复。

为了利用当前模型能力，模型可以使用现有 toolCall 机制表达意图。toolCall 进入 Runtime 后统一归一化为协议对象。

4. Tool Call 归一化为 Action

Tool call 先归一化为 Action，再通过 Runtime 获得执行资格。

任何模型侧请求、工具调用、Agent delegation、Runtime operation、human approval、workflow command 或外部 service invocation，在执行前都必须归一化为 Harness `Action` 或 Runtime `Command`，再经过 policy、gate、executor、event、projection。

```
model intent
  -> normalized Action
  -> policy and gate checks
  -> executor invocation
  -> result
  -> event and projection
```

5. 会话上下文由 Runtime 智能构造

Runtime 根据场景动态构造会话上下文，让模型在合适的信息环境中判断和生成输出。

比如可以主动补充记忆、环境信息、当前 Projection、相关 Artifact、约束、依赖和历史证据，还可以对会话中产生的动态内容进行语义解释后生成结构化提示，用于辅助模型理解这些内容是什么原因生成的，代表什么含义。

6. 状态记录与操作视图分离

Runtime 将被接受的状态变更记录为 Event，并从 Event 和状态文件推导当前 Projection。

Projection 是 Runtime、Agent Session 和 UI 的默认操作视图；Event Log 用于审计、回放、调试和恢复。

7. Trace / Observability 是治理证据层

Trace 是 Runtime 从 Event、Action、Assignment、executor invocation、Artifact、Gate、Decision 和 observation 中组织出的可观察证据链。

Observability 是 Harness 治理能力的一部分，用于解释系统为什么执行、执行到了哪里、哪个环节失败、哪些证据支撑当前 Projection，以及如何审计、评测和恢复。

8. 流程能力统一在 Action Graph

Harness 基础 DSL 提供通用 Action Graph 语义。Runtime 接受的执行流程都会持久化为 Action Graph，并使用同一套状态、调度、恢复、Trace 和 Artifact 模型执行。

Workflow 是用户创建、保存或命名后的 Action Graph Profile。平时任务生成的图仍称为 Action Graph；当用户保存为 Workflow 或主动创建 Workflow 时，它成为 Workflow 资产。

Retry、loop、gate、handoff、verification、decision 和恢复可以由 Action Graph 字段表达，也可以由 Agent metadata 的 Orchestration Policy 或 Runtime policy 补齐，最终都会归一化为统一 Action、状态迁移和事件投影。

9. 协议产物面向人和 Agent 可读

Artifact、Trace、Projection 和 UI 视图都需要提供稳定 id、结构化摘要、状态、引用、来源、证据和可见性信息。

这些产物既服务人类观察和治理，也服务后续 Agent Session 读取、接续、审查和复用。

协议架构

协议架构说明 Harness 的协议构件、运行平面和分层边界。

协议构件

Harness 协议由七类协议构件协同工作：

- **Model**：负责判断、规划、解释和产生产物，通过模型-Runtime 协议声明意图。
- **Runtime**：负责状态、权限、调度、执行、门禁、持久化、审计和恢复。
- **Agent**：可复用执行模板。Runtime 基于 Agent 模板创建会话，由会话接收 assignment，并在授权上下文中完成具体工作。Agent 模板可以声明默认 Orchestration Policy，供 Runtime 主动评估前置准备、完成后验证、失败恢复、风险审查和冲突仲裁。
- **State**：通过 Event Log、Projection、Trace、Artifact、Memory 和 Concept 表示可审计系统状态。
- **Workflow Profile**：用户创建、保存或命名后的 Action Graph Profile，用于复用、管理和从模板启动新的 Run。
- **Adapter**：在 Harness 协议之上实现 evaluation loop、release gate、long-running automation 等产品接入形态。
- **UI**：负责观察、管理、批准、暂停、恢复和追溯，通过 Command 推进系统。

运行平面

运行平面说明协议构件如何沿运行链路协作，覆盖用户输入、模型声明、Runtime 归一化、执行、状态持久化、投影更新和产品观察。

由五个平面组成。

Model interaction plane

Model

-> Model-Runtime Protocol

-> Action declaration / answer / recovery input

Execution plane

Runtime

-> Normalize

-> Validate

-> Route

-> Gate

-> Assign

Executor

-> Tool / Agent Session / Runtime Service / Human / Pipeline

State plane

```
-> Event Log
-> Projection
-> Trace
-> Artifact
-> Concept
```

Context construction plane

```
-> Context Bundle
-> Memory
-> Visibility Policy
```

Adapter and product access plane

```
Workflow Profile -> saved / created Action Graph assets
Adapter -> Evaluation loop / Release gate / Long-running automation
UI -> Command / Approval / Observation
```

模型交互平面

模型与 Runtime 的交互由 `02-model-runtime-protocol.md` 定义。

该协议定义了：

- 模型如何声明 Action、依赖、上下文引用和结果策略？
- 当模型不遵循协议仍然输出 toolCall 时，Runtime 如何把直接 tool request 安全恢复为协议 Action？
- Runtime 如何把结果重新呈现为精简 observation，而不仅仅是原样塞回会话？
- 如何兼容模型的特点，利用 toolCall 机制来实现协议。

执行平面

执行平面由 `03-action-executor-contract.md`、`04-routing-and-delegation-policy.md` 和 `05-handoff-protocol.md` 定义。

- **Action**：Runtime 接受后的可执行语义单元，描述要执行什么操作、携带什么参数、涉及哪些资源、有什么 side effect、期望什么结果返回策略，以及建议由哪类 executor 执行。所有模型请求、toolCall、delegation request、runtime operation 或 human approval 在执行前都归一化为 Action。
- **Executor**：执行 Action 的统一抽象，表示由哪类执行者完成这个 Action。具体目标可以是 tool、Agent Session、runtime service、human、pipeline 或 external service。Executor 的执行环境契约覆盖 Sandbox、Workspace、Manifest、Snapshot 和 Rehydration。
- **Routing**：Runtime 把 Action 绑定到具体 executor 的选择过程。Routing 根据 operation、executor hint、资源范围、side effect、权限、可用性、成本、agent entry/capability 和当前 run state 做选择，并记录选择结果。
- **Delegation**：Routing 选择 Agent Session 作为 executor 时形成的执行形态。Runtime 基于 Agent 模板创建会话，生成 Assignment，并记录 child session trace；Assignment 是 Action 绑定给 Agent Session 后形成的执行边界。

- **Handoff**: Assignment 或 Agent Session 到达交接边界时, Runtime 把来源会话的结果、证据、风险、未决项和 raw refs 归一化为 Handoff record, 并渲染为目标 Context Bundle。

状态控制平面

状态控制平面定义 Harness 中状态如何被记录、投影、读取、同步和恢复, 由 `06-state-event-projection-model.md` 定义。

它通过以下对象和机制控制状态:

- **Event**: 状态变更记录, 说明发生了什么, 用于审计、回放和恢复。
- **Projection**: 当前操作视图, 由 Event 和状态规则推导, 供 Runtime、Agent Session 和 UI 读取。
- **Trace / Observability**: 围绕 run、action、assignment、executor、artifact、gate、decision 和 observation 组织出的可观察证据链, 供 UI、审计、调试、评测和恢复使用。
- **Materialized state**: 状态的持久化形态, 例如数据库行、状态文件、Profile JSON、索引文件和 UI summary, 用于快速查询、展示、调度和恢复。
- **Canonical status**: 统一状态词汇, 例如 `ready`、`running`、`blocked`、`partial`、`completed`, 用于对齐 run、action、assignment、adapter 的状态含义。
- **Transaction / replay / export**: 状态控制机制, 定义状态如何安全写入、如何从历史重建、如何导出为可审计 trace。

上下文构造平面

上下文构造平面定义 Runtime 如何根据当前场景组织模型输入, 由 `07-context-memory-visibility-policy.md` 定义。

它通过以下对象和机制运行:

- **Context Bundle**: 一次模型调用或 assignment 的结构化输入包。Runtime 将当前会话内容、Projection、环境信息、Artifact refs、约束、依赖、相关 Memory 和语义解释提示组织到 Context Bundle 中, 供模型判断和生成输出。
- **Memory**: 上下文构造的一类内容来源。Runtime 可以从 run、project、team、global 等 scope 检索相关记忆, 并以带 scope、status、evidence refs 和 freshness 信息的记录形式加入 Context Bundle。
- **Visibility Policy**: 定义同一份信息如何进入不同通道, 例如 model context、UI、logs、future memory 或 runtime-only control。Runtime 根据 visibility 决定内容是摘要、结构化提示、引用、完整输出还是仅作为内部控制信息使用。
- **Semantic interpretation**: Runtime 可以对会话中产生的动态内容生成结构化解释, 说明这些内容是什么原因生成的、代表什么含义、与当前 Projection 或历史证据有什么关系, 用于辅助模型推理。

Adapter 与产品接入平面

Workflow Profile 由 `08-workflow-profile-action-graph-assets.md` 定义。UI 由 `09-ui-console-and-agent-management.md` 定义。

它们回答:

- Workflow 如何作为被用户创建、保存或命名的 Action Graph Profile?
- 任务执行产生的 Action Graph 与 Workflow Run 如何使用同一套持久化、恢复和 UI 投影?
- UI 如何展示 run、task、decision、event、memory、concept、agent manager 和 session tree?

- 用户如何通过 Command 推进系统状态？

分层边界

分层边界用于确定责任归属、状态归属和跨层接口。子协议设计字段、状态机和 UI 行为时，应沿用这组归属关系。

Layer	拥有内容	跨层接口
Model Layer	判断、计划、协议声明、用户可见解释	协议输出、DSL 声明、回答、恢复输入
Runtime Layer	状态迁移、权限校验、调度、门禁、事件追加、投影更新、恢复	Command、Action、Assignment、Event、Projection
Execution Layer	工具、Agent 会话、Runtime 服务、人工批准、流水线、外部服务	执行器调用、执行结果、产物
State Layer	Event Log、Projection、Trace、Artifact、Memory、Concept、Profile 状态、Adapter 状态	状态查询、上下文引用、trace/export 引用
Adapter Layer	Workflow 资产、evaluation loop、release gate、long-running automation	场景化资产、操作和状态投影
Product Layer	Harness Console、Session Tree、Agent Manager、Protocol Panel	UI 命令、批准、观察、trace 视图

控制流

Harness 的标准执行链路是：

User request

-> Runtime creates or updates Run intent

-> Model declares answer or protocol output

-> Runtime returns answer, or normalizes protocol output into Action

-> Runtime validates policy and gates

-> Runtime routes Action to Executor

-> Runtime creates Assignment when delegating to Agent Session

-> Executor returns result

-> Runtime stores artifact and appends Event

-> Runtime updates Projection

- > Model receives concise observation when needed
- > UI observes Projection and trace

关键点：

- 模型输出先进入 Runtime normalization。
- 工具结果通过 Runtime observation policy 进入下一轮模型上下文。
- Executor 是执行 Action 的目标；Assignment 是 Action 被委派给 Agent 会话时形成的任务边界。
- Agent delegation 通过 Runtime Assignment 发生。
- UI 中会改变系统状态的操作必须提交为 Command，由 Runtime 校验、执行并记录。
- Workflow Profile、Adapter 状态和产品视图必须投影回统一 Harness run state。

决策边界与升级规则

Harness 按风险、影响半径、可逆性和价值判断含量确定决策边界。Runtime 可以直接处理确定性执行；局部可逆执行可以交给授权执行者；影响目标、约束、架构、长期概念或 owner 偏好的决策需要升级。

分级如下：

Level	含义	典型处理
L0	确定性执行	Runtime 或 deterministic tool 直接处理。
L1	局部可逆执行	授权执行者可在授权 scope 内处理。
L2	影响局部工程质量	授权评估者或决策者审查后推进。
L3	影响目标、约束、架构或长期概念	授权决策者必须显式记录理由。
L4	影响 owner 偏好、成本、风险或不可逆外部后果	需要 human owner 或等价授权。

子协议可以细化 gate，但应保持同一套升级语义。

Workflow 定位

Workflow 是用户创建、保存或命名后的 Action Graph Profile。

Harness 的执行能力统一属于 Action Graph 和 Runtime。任务执行时生成的图称为 Action Graph；用户将其保存为 Workflow，或用户主动创建 Workflow 后，它成为可管理、可复用的 Workflow 资产。

所有 Action Graph 都是持久化执行对象，支持：

- 多阶段任务

- 并行子任务
- 多 Agent 执行
- 系统重启后恢复
- pause / resume / abort
- retry、loop、replan、decision
- gate、verification、approval
- Artifact、Trace、Projection 和 UI 观察

Workflow Run 从 Workflow Profile materialize 出新的 Action Graph，并使用与任务执行生成的 Action Graph Run 相同的 Action、Assignment、Event、Projection、Trace 和 Artifact 模型。

UI 定位

UI 是 Harness 治理能力的操作面。

UI 读取 Projection、Event、Artifact、Trace、Memory 和 Concept。UI 中会改变系统状态的操作提交 Command，由 Runtime 执行状态变更。

UI 视图、Trace export 和 Artifact summary 需要保持 agent-readable：提供稳定引用、结构化摘要、状态、来源、证据和可见性信息，让人类和后续 Agent Session 都能理解和复用。

UI 需要展示：

- run 当前状态
- task 和 assignment 进度
- blocker 和 pending decision
- gate 失败原因
- artifact 和 evidence
- child session trace
- memory 和 concept 的来源
- agent 配置、entry、capability 和 permission

共享字段与状态词

统一字段

模型 DSL、Action、Action Graph Profile、Assignment Contract、Handoff Contract、Workflow Profile 和场景化 Adapter Profile 使用同一组治理字段。字段名保持短、稳定、模型友好；概念解释保留完整语义名称。

字段	语义对象	用途
<code>depends_on</code>	Action Graph Edge	声明当前工作依赖哪些上游 call、Action、node、Artifact 或 Decision。

字段	语义对象	用途
<code>criteria</code>	Success Criteria	声明完成标准，供 Runtime、Executor、Gate、Review、UI 判断是否完成。
<code>failure</code>	Failure Policy	声明失败、阻塞或验证未通过后的处理路径，例如 retry、block、ask_user、handoff、abort。
<code>budget</code>	Budget Policy	声明成本、时间、token、attempt、parallelism、缓存和外部资源约束。
<code>visibility</code>	Visibility Policy	声明结果进入 model context、user UI、logs、trace、future runs 或 runtime-only control 的方式。
<code>artifacts</code>	Artifact Contract	声明期望产生、读取、更新或引用的 Artifact 类型、名称、scope、visibility 和 evidence 要求。
<code>handoff</code>	Handoff Contract	声明下游交接目标、约束、依赖、证据、Artifact、预算、风险和未决问题。
<code>context</code>	Context Request	声明需要 Runtime 选择、展开或摘要的 memory、artifact、projection、trace 或 environment refs。
<code>gate</code>	Gate Policy	声明 approval、verification、review、privacy、release gate 等状态迁移条件。
<code>result</code>	Result Policy	声明执行结果回放给模型、用户、日志和 Artifact store 的粒度。

这些字段在不同层表达同一语义。模型侧可以只声明其中一部分，并可使用 string、array 等简写形态；Runtime 在归一化时根据 Projection、authority、routing、profile policy、adapter policy 和 safety policy 展开为对象形态，补齐可执行边界。

规范状态词

Run、Action Graph、Action、Assignment、Workflow Run、Workflow asset、Adapter run 和 UI Projection 使用同一组状态词。

Status	语义
<code>draft</code>	已创建，尚未进入可调度状态。
<code>pending</code>	已接受，但还在等待依赖、输入 materialization 或调度窗口。
<code>ready</code>	合法且可调度。
<code>running</code>	正在执行。
<code>waiting_user</code>	等待用户或 Owner 输入。
<code>waiting_permission</code>	等待权限、审批或授权。
<code>blocked</code>	缺少决策、前置条件、权限、证据或资源，无法继续。
<code>partial</code>	已产生可用结果，但部分必要子项、验证、handoff、Artifact 或 gate 未完成或未通过。 <code>partial</code> 不是 <code>completed</code> ，下游只能消费被 Runtime 明确标记为可用的 Artifact，并需要通过 failure、Decision 或 Handoff 处理未决部分。
<code>failed</code>	执行失败，当前 <code>failure</code> 未恢复。
<code>completed</code>	成功完成，满足 criteria 和必要 gate。
<code>skipped</code>	被 Decision、Gate 或依赖规则跳过，且跳过行为已记录。
<code>cancelled</code>	Runtime 或用户在完成前取消。
<code>aborted</code>	Run 或高层编排被有意终止为最终状态。

对象定义

本节定义 Harness 协议的共享对象。子文档可以细化字段和状态机，但应沿用这些对象语义。

Actor

可以参与协议并出现在事件、命令、审计记录中的主体。Actor 是最宽的执行身份概念，覆盖 Agent Session、Runtime 服务、确定性程序、工具包装器、Profile runner、Adapter runner 和 human owner。

示例：

- `agent-session-13`

- `memory-service`
- `workflow_asset_manager`
- `bun-test-runner`
- `human-owner`

Model

生成回答、协议输出和推理结果的模型能力。Model 可以由不同 provider 或 runtime backend 提供，通常通过 Agent Session 被使用。

Model 不直接改变 Harness 状态；状态变更需要通过 Runtime 接受的 Command 或 Action 发生。

示例：

- 用于主会话的 LLM
- 用于 review session 的 LLM
- 用于 summarization 的模型服务

Agent

可复用执行模板。Agent 定义 persona、prompt material、entry、capability、permission profile、模型偏好、运行策略、relationship metadata 和可选 Orchestration Policy。

Agent 本身不直接执行 assignment。Runtime 基于 Agent 模板创建 Agent Session，由会话执行具体工作。

Orchestration Policy 用于声明 Runtime 可以围绕当前 Agent Session 评估的默认编排策略。Runtime 会将命中的策略展开为标准 Action / Assignment；需要交接给后续 Agent 或 human 时，展开为 Handoff Contract，并为每个后续 Assignment 独立校验权限、预算、gate 和状态。

Agent 模型由 `01-agent-model-and-authoring.md` 定义。

示例：

- 负责开发工作的 `code_developer`
- 负责审查补丁的 `technical_reviewer`
- 负责准备上下文的 memory agent
- 负责 Workflow 资产创建、保存、更新和启动的 `workflow_runner`

Orchestration Policy

Agent 模板上的默认 Runtime 编排策略。Orchestration Policy 描述 Runtime 可以根据请求状态、结果状态、Artifact、side effect、风险、阻塞、失败和冲突触发哪些额外 Action / Assignment。

Orchestration Policy 的产物是标准 Contract；涉及跨 Agent 或 human owner 交接时，产物是 Handoff Contract。Runtime 负责判断是否触发、选择目标 Agent、推导 authority、构造 Context Bundle、追加 Event、更新 Projection 和记录 Trace。

示例：

- 需求不清楚时触发 `requirements_clarifier`

- `code_developer` 完成写入后触发 `code_test`
- `code_test` 完成后触发 `technical_reviewer`
- `frontend_developer` 完成 UI 修改后触发 `accessibility_reviewer`
- 测试失败后触发 `code_debugger`
- 多个 Agent 判断冲突时触发 `technical_reviewer` 仲裁

Agent Session

Runtime 基于 Agent 模板创建的执行会话。Agent Session 有会话 id、session log、trace、状态和权限边界，可以作为 root session、child session 或 descendant session 存在。

Agent Session 接收 assignment，生成协议输出、Action 请求、Artifact 或结果摘要。Agent 会话之间不直接通信，协作由 Runtime 通过 assignment、event、projection 和 trace 协调。

Agent Session 不等同于模型上下文。Session log 是会话的持久记录，保存会话中被 Runtime 接受和引用的消息、协议输出、Action、observation、Artifact ref 和状态变化；Context 是 Runtime 在模型调用前基于 session log、Projection、Memory、Artifact、环境信息和语义解释动态构造出的模型输入。

示例：

- root coding session
- review child session
- Workflow asset management session
- memory summarization session

Workflow Profile

用户创建、保存或命名后的 Action Graph Profile。Workflow Profile 记录可复用流程的 graph、输入 schema、版本、owner、visibility 和运行历史。

保存为 Workflow 后，这份 Action Graph Profile 获得名称、版本、输入 schema、权限和 UI 管理入口。Workflow Run 会从 Profile materialize 出新的 Action Graph，并进入统一 Runtime 执行路径。

示例：

- 用户保存的一次发布流程
- 固定代码审查流程
- 常规评测流程模板
- 长期监控流程模板

Adapter

基于 Harness 基础 DSL 的产品接入结构。Adapter 用于表达 evaluation loop、release gate、long-running monitor 等常见场景。

Adapter 定义场景化结构、模板和约束。Runtime 将 Adapter 输入展开为统一 Action Graph、Action、Assignment、状态迁移、事件投影、Trace 和 Artifact。

示例：

- evaluation loop adapter
- release gate adapter
- long-running monitor adapter

Capability

Agent 或 executor 的适配能力描述，用于 routing、selection 和 prompt construction。

Capability 是匹配元数据，不授予执行权限。实际可执行边界由 Authority 决定。

示例：

- `read_code`
- `write_code`
- `run_tests`
- `code_review`
- `query_project_memory`
- `summarize_artifacts`

Authority

Runtime 授予 Actor 在某个 scope 内可以改变、读取、批准或触发什么。Authority 是执行边界，通常绑定到 Action、Assignment 或 Command。

Authority 由 Runtime 强制执行。

示例：

- 读取 `repo://current`
- 写入 `packages/harness/src/workflow/**`
- 创建 review artifact
- 批准 L2 task completion
- 请求 L4 owner decision

Scope

协议对象生效的边界。Scope 用于限定权限、记忆、上下文、artifact 和审计范围。

示例：

- `run`
- `project`
- `team`
- `global`
- `assignment`
- `workflow`

Namespace

Scope 内的命名空间。Namespace 让不同项目、团队或运行上下文中的对象使用统一格式，同时保持身份明确。

示例：

- `project:open-agent-harness`
- `team:runtime`
- `global:user`
- `run:run_123`

Runtime

Harness 的执行控制层。Runtime 负责解析协议、校验权限、选择 executor、创建 assignment、追加事件、更新 projection、执行 gate、构造上下文和管理恢复。

示例职责：

- 接受 `Command`
- 将模型输出归一化为 `Action`
- 选择合适的 `Executor`
- 创建 `Assignment`
- 写入 `Event`
- 重建 `Projection`

Command

用户、UI 或 Actor 提交给 Runtime 的结构化状态变更请求。

Command 经过 schema、authority、gate 和 projection 校验后，才能产生 Event 或 Action。

示例：

- `run.create`
- `run.pause`
- `task.retry`
- `decision.answer`
- `verify.rerun`

Action

Runtime 接受后的可执行语义工作单元。

Action 可以映射到 tool、Agent Session、runtime service、human approval、pipeline、Workflow Command 或 service invocation。Action 契约由 `03-action-executor-contract.md` 定义。

示例：

- 读取文件

- 搜索代码
- 委派 review agent session
- 请求 human approval
- 从 Workflow Profile 启动 run

Action Graph

由 Action nodes 和 dependency edges 组成的执行图。Action Graph 表达哪些工作可以并行、哪些工作依赖上游结果、哪些 gate 或 handoff 决定下游是否可继续。

Action Graph 可以来自模型侧 `calls[]`，也可以从 Workflow Profile、Evaluation Profile、Release Gate Profile 等场景化 profile materialize。Runtime 负责持久化 graph、校验依赖、调度 ready actions、记录状态、处理失败并更新 Projection。

示例：

- `inspect -> implement -> test -> review`
- `read_a + read_b -> compare -> summarize`
- `implement -> [unit_test, typecheck] -> gate`

Executor

执行 Action 的目标抽象。Executor 可以是 tool、Agent Session、Runtime service、human、pipeline 或 external service。

示例：

- `tool:read`
- `tool:grep`
- `agent-session:reviewer`
- `runtime:summarize`
- `human:owner_approval`

Assignment

Runtime 将 Action 委派给 Agent Session 后形成的任务边界。

Assignment 包含 action refs、agent session refs、authority、input contract、output contract、context refs、artifact refs 和 trace refs。

示例：

- 把 `review_changes` action 绑定给 reviewer session
- 把 `implement_fix` action 绑定给 coding session
- 把 `summarize_context` action 绑定给 memory session

Contract

Assignment 或 Action 的输入输出契约。Contract 定义执行者收到什么、要达成什么目标、遵守什么约束、依赖哪些对象、提交什么 Artifact、提供什么证据、消耗什么预算、披露什么风险、遗留哪些未决问题，以及结果如何被 Runtime 验收。Contract 使用总纲定义的统一字段表达这些边界。

示例：

- 目标：完成指定 review、实现或验证任务
- 约束：scope、权限、时间、成本、风格、兼容性
- 依赖：`depends_on` 中的 upstream action、artifact refs、decision refs
- 输出：result summary、artifact refs、status
- 证据：test log、diff artifact、review finding
- 风险：已知失败、未验证路径、scope drift
- 未决问题：需要 owner、reviewer 或后续 Agent Session 继续判断的事项
- 验收：status、summary、gate result

Success Criteria

Action、Assignment、Run 或 Action Graph node 的完成标准。Success Criteria 让 Runtime、Executor、Review、Verification 和 UI 可以围绕同一组验收条件判断工作是否完成。

示例：

- “焦点测试通过。”
- “Review finding 都有 evidence 和 severity。”
- “发布 gate 所需审批全部完成。”

Failure Policy

执行失败、阻塞或验证未通过后的处理规则。Failure Policy 可以描述 retry、block、ask_user、handoff、abort、continue_with_warning 等路径，并受预算、权限和 gate 约束。

示例：

- retry max attempts 2
- 失败后交给 debugger Agent Session
- review 未通过时阻塞 parent run

Budget Policy

Runtime 对成本、时间、token、attempt、parallelism、缓存和外部资源的约束。Budget Policy 可以绑定到 Run、Action、Assignment、Workflow Profile、Adapter 或 Model Policy。

示例：

- `timeout_ms: 600000`
- `max_attempts: 2`
- `max_parallel: 3`

- `cache: read_allowed`

Visibility Policy

控制信息进入不同通道的方式。Visibility Policy 规定 Artifact、Trace、Action result、Memory 或 Observation 如何进入 model context、user UI、logs、future runs 和 runtime-only control。

示例：

- model: summary
- user: summary
- logs: full
- future_runs: ref

Handoff

Runtime 在 Assignment 之间建立的结构化交接关系。Handoff 用于把一个 Agent Session 的结果、证据、风险和未决问题交给另一个 Agent Session 或 human owner 继续处理。

Handoff 包含目标、约束、依赖、证据、Artifact refs、预算、风险、未决问题、来源 session 和目标 executor，并使用 `handoff` 字段进入 Action、Assignment 或 Action Graph node。

Handoff 可以来自模型声明的 next Action，也可以来自 Agent 模板上的 Orchestration Policy。无论来源如何，Runtime 都将其转换为受治理的 Action / Assignment。

`05-handoff-protocol.md` 进一步定义 Handoff 的生成流程：Runtime 保存 raw records 和 structured trace，在边界处请求 source Agent 生成 self-report，handoff writer 做压缩整理，Runtime 最后校验并渲染给目标 Context Bundle。

示例：

- coding session 完成 patch 后交给 review session
- review session 提出 rework 后交给 coding session
- verification session 发现 blocker 后交给 human owner 决策

Event

Runtime 接受并追加的不可变事实。

事件是审计、回放和恢复基础。模型上下文通过 Projection、Context Bundle 和 observation 获得。

示例：

- `run.created`
- `action.accepted`
- `assignment.started`
- `artifact.written`
- `gate.blocked`
- `decision.applied`

Event Log

Event 的追加式记录。Event Log 是审计和 replay 的事实来源，回答“系统接受了什么、何时接受、由谁提交、Runtime 为什么接受”。

示例用途：

- 回放 run 状态
- 解释一次 gate block
- 导出审计材料
- 恢复 projection

Projection

由 Event 和状态规则推导出的当前操作视图。

UI、Runtime 调度、Agent Session 上下文都默认读取 Projection。

示例：

- run 当前状态
- task 当前状态
- pending decision queue
- artifact index
- concept current version
- memory index

Trace

围绕一次 run、action、assignment 或 Agent Session 组织出的可观察证据链。Trace 连接 Event、Projection、executor invocation、Artifact、observation、Gate、Decision 和 error，用于审计、调试、评测、恢复和后续 Agent Session 接续。

示例：

- protocol action trace
- child session trace
- executor invocation trace
- gate failure trace
- audit export trace

Materialized State

Projection 或场景状态的持久化形态，用于快速查询、展示、调度和恢复。

示例：

- 数据库里的 run/action/assignment 行
- action graph node state 文件
- Workflow Profile state 文件

- artifact index
- UI summary cache

Trigger

Runtime 根据 Event 和 Projection 执行的确定性规则。Trigger 把已接受事实转化为下一步系统动作。

示例：

- task submitted 后触发 review assignment
- review approved 后触发 verification
- verification failed 后触发 rework decision
- decision answered 后恢复 blocked run

Gate

状态迁移或执行前需要满足的 Runtime 强制条件。

Gate 可以校验 schema、authority、scope、dependency、review、verification、human approval、privacy 和 budget。

示例：

- schema valid
- write scope allowed
- required review approved
- verification passed
- human approval granted
- sensitive output redacted

Run

一次受 Harness 管理的工作执行。Run 聚合目标、任务、Action、Assignment、Artifact、Event、Projection、Decision 和 trace。

示例：

- 修复一个 bug 的 run
- 一次代码审查 run
- 一次从 Workflow Profile 启动的 run
- 一次 release gate run

Task

Run 内的可管理工作单元。Task 表达用户或 Planner 关心的工作拆分，可以由一个或多个 Action、Assignment、Artifact 和 gate 组成。

示例：

- `inspect_auth`

- `implement_timeout`
- `review_patch`
- `verify_sdk_build`

Artifact

由 Actor 或 Runtime 生成、存储、引用的产物。Artifact 用引用进入上下文，避免把完整内容反复放入模型输入。

示例：

- diff patch
- test log
- review report
- generated plan
- protocol trace export
- Action Graph node result

Artifact Contract

Action、Assignment 或 Action Graph node 对产物的声明。Artifact Contract 定义期望产生、读取或更新的 Artifact type、name、scope、visibility 和 evidence 要求。

示例：

- 期望 `patch` artifact
- 期望 `test_report` artifact
- 读取 upstream `review_report`

Environment Manifest

Runtime 为 Executor 创建的环境声明。Environment Manifest 描述 executor 的 sandbox、workspace、工具、网络、secret、artifact 输出、snapshot 和 timeout。

示例：

- 只读 workspace manifest
- 可写临时 worktree manifest
- 带 snapshot refs 的恢复 manifest

Context Bundle

Runtime 为一次模型调用或 assignment 构造的上下文包。

它包含当前会话内容、目标、约束、当前 Projection 摘要、必要 Artifact refs、Memory refs、环境信息和语义解释提示。

示例内容：

- task goal

- relevant files
- accepted decisions
- artifact refs
- current projection summary
- memory refs

Memory

跨时间保存的知识或偏好。

Memory 记录需要包含 scope、status、source、visibility 和 evidence。Projection 提供当前操作真相，Memory 提供可检索证据。

示例：

- 项目命名约定
- 团队 review policy
- 用户偏好的输出格式
- 架构决策摘要

Concept

系统需要长期稳定引用的概念，例如 API 名、schema、module ownership、architecture decision。

Concept 可以记录版本、替换关系、引用、证据和影响范围。Concept replacement 需要说明新的事实、需求、失败或约束，并触发 impacted reference scan。

示例：

- `user-profile-api`
- `agent-entry-model`
- `action-graph-node-state`
- `runtime-permission-policy`

Goal Contract

Run 的目标边界。Goal Contract 记录目标、non-goal、约束和成功标准，让 Runtime 和授权决策者判断目标漂移。

示例：

- goal: “修复 auth refresh timeout”
- non-goal: “重写 auth provider”
- constraint: “保持 public API”
- success: “typed timeout error + focused tests pass”

Decision

对不确定性、风险、目标变化或恢复路径的结构化选择。Decision 由 Runtime 接受并记录，通常带有 reason、scope、evidence 和 actor。

示例：

- 选择 retry node
- 批准 L2 task completion
- 将概念替换升级到 L3
- 请求 human owner approval

Review

对提交产物的评估。Review 关注正确性、回归风险、遗漏测试、scope drift、已披露风险和证据质量。

示例输出：

- finding
- severity
- evidence
- recommendation
- decision: approve / request_rework / needs_info

Verification

通过工具、测试、脚本或确定性检查验证声明。Verification 把“看起来完成”转成可引用证据。

示例：

- `bun typecheck`
- focused unit test
- schema validation
- replay check
- smoke test

Escalation

把决策提升到更高权限或更大影响半径处理。Escalation 依据风险、可逆性、成本、目标漂移和 owner preference。

示例：

- L2 task issue 升级为 L3 architecture decision
- 发布前 gate 升级给 owner approval
- concept replacement 触发 impact scan 后升级

Owner

对某类高影响决策拥有最终授权的人类或组织主体。Owner 可以是用户、项目维护者、团队负责人或组织策略。

示例：

- user owner
- repository maintainer
- release owner
- security owner

待讨论问题

这些问题需要继续讨论，并在对应子协议中定义：

- direct request recovery 的精确判定规则。
- Agent routing 的 scoring 和 tie-breaker。
- Memory ranking 与 historical evidence 返回规则。
- Concept replacement 的 evidence 和 impact scan 规则。
- UI audit/export 的 redaction policy。
- Model Policy：模型选择、成本预算、fallback、缓存和调用策略。
- Evaluation Adapter：Trial、Grader、Outcome、Metric 和 Regression Gate。

文档地图

编号	文档	责任
00	00-harness-governance-protocol.md	总纲：目标、原则、协议架构、对象定义和边界。
01	01-agent-model-and-authoring.md	Agent 模板、Skill 来源导入、metadata、entry、capability、permission、relationships、authoring contract。
02	02-model-runtime-protocol.md	模型与 Runtime 的 DSL / protocol 交互协议。
03	03-action-executor-contract.md	Action、Executor、toolCall carrier、recovery normalization、执行环境契约。
04	04-routing-and-delegation-policy.md	Routing、delegation、assignment、child session trace 和 executor selection。

编号	文档	责任
05	<code>05-handoff-protocol.md</code>	Handoff 触发、self-report、source bundle、handoff writer、canonical record 和目标 Context Bundle 渲染。
06	<code>06-state-event-projection-model.md</code>	Event source、Projection、Trace / Observability、状态映射、事务边界、replay。
07	<code>07-context-memory-visibility-policy.md</code>	Context Bundle、Memory Scope、visibility、privacy、replay policy。
08	<code>08-workflow-profile-action-graph-assets.md</code>	Workflow Profile、Workflow asset 与统一 Action Graph 执行语义。
09	<code>09-ui-console-and-agent-management.md</code>	UI 如何管理、观察和使用 Harness 系统。

支持性研究、评测材料和参考资料放在 `docs/harness-protocol/support/`，作为协议设计依据和评测背景。

RFC、实现计划和阶段性执行编排不放在协议目录中。它们归入 `docs/harness-rfc/`：

文档	责任
<code>../harness-rfc/01-agent-metadata-control-plane-rfc.md</code>	Agent metadata control plane RFC。
<code>../harness-rfc/02-agent-metadata-implementation-plan.md</code>	Agent metadata RFC 的实现计划、测试边界和 UI/API 落点。
<code>../harness-rfc/03-agent-metadata-execution-orchestration.md</code>	Agent metadata 实施编排：phase、并行窗口、PR 边界和 worker handoff。

Agent 模型与编写规范

Agent 模板存放在 `config/agents/<id>/` 下，包含 `meta.json`，并可选包含 `identity.md` 和 `rules.md`。`<id>` 目录名必须与 `meta.json.id` 一致；package 模板和用户模板合并后，所有 id 必须唯一。

本文定义 Harness 的 Agent 编写协议。Agent 是可复用模板，包含入口规则、能力元数据、prompt 材料、模型偏好和权限策略。具体工作需要执行时，Runtime 从模板创建 Agent Session。

Runtime 将 package、user、project 三类模板加载到同一个 registry 中。相同 id 的后加载模板覆盖先加载模板。无效模板产生诊断信息，但不阻止有效模板加载。

Harness 区分三类信息：

- `entry`：Agent 可以从哪里被调用。
- `capability`：Agent 适合做什么。
- `permission`：Agent 在某个具体 assignment 中实际可以做什么。

Agent metadata 是协议对象，不只是 UI 展示信息。Runtime 需要通过 metadata 理解一个 Agent 的身份、能力、入口、权限、依赖、成本、风险和可观测性边界，才能在不同场景下做 routing、delegation、context construction、permission derivation、trace、evaluation 和 recovery。

清晰的 Agent metadata 解决四类问题：

- **管理**：系统知道哪些 Agent 可见、可委托、可作为主 Agent、默认是否可选，以及在什么条件下应被隐藏或禁用。
- **调度**：系统根据 capability、成本、写入倾向、模型偏好、依赖和编排策略选择合适 Agent，而不是依赖自然语言猜测。
- **隔离**：每个 Agent 拥有自己的 session、authority、context、artifact 和 trace，方便独立调优、测试、评估和替换。
- **协作**：Agent 之间的关系通过 Runtime 表达为 Action、Assignment、Handoff Contract 和 Event，而不是两个 session 直接互相发消息。

因此，Agent 定义至少需要覆盖这些 metadata 面：

- identity metadata: `id`、`name`、`description`、`persona`
- entry metadata: `entry`
- capability metadata: `capability.purpose`、`capability.tags`、`capability.cost`、`capability.writes`
- permission metadata: `permission_mode`、`allowed_tools`、`denied_tools`、`inherit_permissions`
- runtime metadata: `model_preference`、`execution_mode`
- relationship metadata: `relationships`
- orchestration metadata: `orchestration_policy`

`relationships` 描述 Agent 与其他 Agent 或 `capability` 的稳定关系。它帮助 Agent Manager、`catalog`、`routing prompt` 和评估系统理解 Agent 的协作结构。真正触发执行时，Runtime 仍应通过 `orchestration_policy` 或显式 Action 创建 Assignment。

Agent 模板与 Agent Session

Agent 模板本身不执行 assignment。Runtime 从 Agent 模板创建 Agent Session，将 assignment authority 绑定到该 session，并把 session 记录为 run trace 的一部分。

一个 Agent Session 包含：

- 稳定的 session id
- 来源 Agent 模板 id
- assignment 或 root run 关系
- session log
- 生效 authority
- trace refs
- status

Session log 是与该 session 相关的已接受消息、协议输出、Action、Observation、Artifact refs 和状态变更的持久记录。

Model context 与 session log 不是同一个对象。每次模型调用前，Runtime 根据 session log、当前 Projection、相关 Memory、Artifact refs、环境信息、约束和语义解释提示构造 Context Bundle。

必填字段

- `id`：稳定模板 id。使用与目录名相同的值。会裁剪空白字符，空值会被拒绝。
- `name`：展示名称。会裁剪空白字符，空值会被拒绝。
- `persona`：用于 prompt 构造的简短行为契约。会裁剪空白字符，空值会被拒绝。
- `description`：面向用户的摘要。会裁剪空白字符，空值会被拒绝。

命名规范

Agent id 使用直观的 `lower_snake_case`，形态为 `<domain>_<work_type>`。

`domain` 表示 Agent 面向的工作领域，例如 `frontend`、`backend`、`database`、`security`、`workflow`、`release`、`session`。`work_type` 表示 Agent 在该领域处理的工作类型，例如 `developer`、`test`、`reviewer`、`planner`、`runner`、`maintainer`、`researcher`、`debugger`、`summarizer`。`work_type` 是命名后缀，用于提高可读性；Runtime routing 仍根据 `capability`、`entry`、`authority`、`availability` 和 `budget` 选择 Agent。

命名示例：

- 开发实现: `frontend_developer`、`backend_developer`、`database_developer`、`code_developer`
- 测试验证: `code_test`、`release_test`
- 审查评估: `security_reviewer`、`api_contract_reviewer`、`performance_reviewer`
- 规划编排: `task_planner`、`plan_executor`、`workflow_runner`
- 研究检索: `code_researcher`、`external_researcher`
- 系统任务: `session_compactor`、`session_summarizer`、`session_title_writer`

展示名称 `name` 可以使用自然语言标题, 例如 `Frontend Developer`。`capability.purpose` 继续作为 routing hint, 不承担命名职责。

可选字段与默认值

- `model_preference`: `{ "providerID": "...", "modelID": "..." }`。当 provider/model catalog 可用时, 两个 id 都必须存在。
- `hidden`: 默认 `false`。
- `execution_mode`: 默认 `auto`。
- `allowed_tools`: 默认 `[]`。
- `denied_tools`: 默认 `[]`。
- `inherit_permissions`: 默认 `false`。Agent 默认使用自身 permission profile; 只有显式设为 `true` 时才继承或合并外部权限策略。
- `permission_mode`: 默认 `strict`。
- `relationships`: 默认不存在。用于声明该 Agent 与其他 Agent 或 capability 的稳定协作关系, 例如上游依赖、推荐下游、互斥关系或替代候选。
- `orchestration_policy`: 默认不存在。用于声明 Runtime 可以围绕该 Agent Session 评估和创建的前置、后置、恢复、审查、仲裁等后续 Action / Assignment。

Entry 模型

`entry` 是 Agent 可被使用位置的事实来源。它刻意与权限和调度质量分离。

```
{
  "entry": {
    "primary": true,
    "delegable": true,
    "mentionable": true,
    "default": true,
    "hidden": false
  }
}
```

字段含义：

- `primary`：可以作为主会话 Agent 运行，并出现在主 Agent 切换器中。
- `delegable`：可以被任务/委托机制启动。
- `mentionable`：可以通过用户 mention 或自动补全直接调用。
- `default`：可以被默认 Agent 解析逻辑选中。
- `hidden`：即使其他 entry flag 为 true，也不应出现在常规用户选择器中。

选择规则：

- 主 Agent 切换器：`entry.primary && !entry.hidden`
- mention 自动补全：`entry.mentionable && !entry.hidden`
- delegation 候选：`entry.delegable && !entry.hidden`
- 默认 Agent 资格：`entry.primary && entry.default && !entry.hidden`

Capability 模型

`capability` 描述 Agent 擅长什么。Dispatcher 和 prompt builder 应结合这些元数据与 `description` 判断何时委托。

```
{
  "capability": {
    "purpose": "architecture_review",
    "tags": ["architecture", "debugging", "review"],
    "cost": "high",
    "writes": false
  }
}
```

字段含义：

- `purpose`：简短稳定的 routing hint。
- `tags`：用于 prompt 生成和 dispatch table 的可搜索能力标签。
- `cost`：相对执行成本，取值为 `low`、`medium` 或 `high`。
- `writes`：声明式元数据，表示该 Agent 是否预期会修改 workspace 状态。

`capability.writes` 是元数据，不是权限 enforcement。实际写入权限仍来自 permission policy。

Execution Mode

`AgentTemplate.execution(meta)` 根据所选模式返回 runtime behavior object：

- `auto`：`{ "autonomous": true, "prompt": false, "review_tools": false, "review_state": false }`。运行常规自主 prompt loop。

- `manual`: { "autonomous": false, "prompt": true, "review_tools": true, "review_state": true }。在执行实质操作前等待明确用户指令。
- `supervision`: { "autonomous": true, "prompt": false, "review_tools": true, "review_state": true }。可以自由计划和检查，但工具使用和状态变更工作需要升级审查。

Permission 模式

`AgentTemplate.permission(meta)` 根据所选模式返回可消费的 permission profile。

Agent 默认不继承项目或上层会话权限。每个 Agent 应拥有自己的权限 profile，Runtime 再根据 assignment authority、run policy、用户审批和 gate requirement 推导最终可执行边界。

- `strict`: 使用 Runtime 定义的 Agent strict profile，并应用 `denied_tools`。当 `inherit_permissions` 为 true 时，可以与项目或上层默认策略合并，但拒绝项优先。
- `lax`: 使用 Runtime 定义的 Agent lax profile，并应用 `denied_tools`。当 `inherit_permissions` 为 true 时，可以与项目或上层允许策略合并，但拒绝项优先。
- `custom`: 使用 `allowed_tools` 和 `denied_tools` 作为作者定义的 Agent permission profile。只有当 `inherit_permissions` 为 true 时，才与外部权限策略合并。

Relationship Metadata

`relationships` 描述 Agent 的协作关系。它让系统知道某个 Agent 经常依赖哪些上游 Agent、常把结果交给哪些下游 Agent、与哪些 Agent 不应并行运行，以及当目标 Agent 不可用时可以考虑哪些替代能力。

Relationship metadata 结构：

```
{
  "relationships": {
    "upstream": [
      {
        "id": "amazon_product_analysis",
        "target": {
          "executor": "agent",
          "capability": "amazon_product_analysis"
        },
        "required": true,
        "order": "sequential",
        "reason": "Cross-border market research needs a product analysis artifact before market synthesis.",
        "artifacts": ["product_analysis_report"]
      },
    ],
    "downstream": [
      {
```

```

    "id": "market_research_synthesis",
    "target": {
      "executor": "agent",
      "capability": "market_research"
    },
    "reason": "Use product analysis as input for broader market research."
  },
  "alternatives": [
    {
      "target": {
        "executor": "agent",
        "capability": "product_research"
      },
      "reason": "Fallback when the Amazon-specific agent is unavailable."
    }
  ],
  "conflicts": [
    {
      "target": {
        "executor": "agent",
        "capability": "destructive_data_migration"
      },
      "reason": "Do not run market research in the same assignment chain as destructive migration."
    }
  ]
}

```

字段含义：

- **upstream**：当前 Agent 执行前通常需要的输入来源。适合声明可复用的前置 Agent，例如跨境调研 Agent 依赖亚马逊商品分析 Agent。
- **downstream**：当前 Agent 产物常交给的后续 Agent。它帮助 UI、catalog 和 Runtime 建议下一步，但不直接授予调用权。
- **alternatives**：目标 Agent 缺失、禁用、成本过高或不可用时可考虑的替代能力。
- **conflicts**：不应在同一 assignment chain 中自动组合的 Agent 或 capability。
- **target**：可以指向具体 Agent id，也可以指向 capability，由 Runtime routing 决定最终 executor。
- **required**：关系是否构成当前 Agent 默认执行质量的一部分。
- **order**：关系展开时建议使用 **sequential** 或 **parallel**。
- **reason**：给 Agent Manager、routing prompt、trace 和评估报告使用的短说明。

- `artifacts`：该关系通常产生或消费的 Artifact 类型。

`relationships` 不等于执行权限。Runtime 可以用它生成候选、解释协作图、辅助评估和补全 `orchestration_policy`，但只有被 Runtime 展开成 Action / Assignment 后才会执行。

如果某个上游关系需要每次执行都自动触发，应在 `orchestration_policy.preflight` 中声明对应编排项。这样元数据负责描述稳定关系，编排策略负责定义触发条件、contract、预算、失败处理和 trace。

Orchestration Policy

Agent 模板可以声明默认 `orchestration_policy`，用于告诉 Runtime：围绕该 Agent Session 的执行过程，哪些状态、风险、结果或 Artifact 变化可以触发额外的 Runtime 编排。

`orchestration_policy` 表达 Runtime 编排策略。Agent Session 仍然只执行自己的 assignment；Runtime 根据策略创建前置准备、完成后验证、失败恢复、风险审查、Artifact 变更响应、冲突仲裁等受治理的 Action / Assignment。

Orchestration policy 结构：

```
{
  "orchestration_policy": {
    "preflight": [
      {
        "id": "clarify_requirements",
        "target": {
          "executor": "agent",
          "capability": "requirements_clarification"
        },
        "required": true,
        "order": "sequential",
        "when": {
          "uncertainty": "high",
          "missing": ["acceptance_criteria"]
        },
        "contract": {
          "goal": "Clarify the implementation goal and acceptance criteria before development.",
          "constraints": ["ask_only_decision_relevant_questions"],
          "depends_on": ["input:user.goal"],
          "evidence": [],
          "artifacts": ["requirements_note"],
          "budget": {
            "cost": "low",
            "timeout_ms": 300000
          },
          "risks": ["ambiguous_scope"],
```



```

        "unresolved": []
    }
}
],
"on_completed": [
    {
        "id": "verify_implementation",
        "target": {
            "executor": "agent",
            "capability": "verification"
        },
        "required": true,
        "order": "sequential",
        "when": {
            "result_status": "completed",
            "requires_artifacts": ["artifact://current/patch"],
            "side_effects": ["write"]
        },
        "contract": {
            "goal": "Verify the implementation result.",
            "constraints": ["use_changed_files", "run_relevant_checks"],
            "depends_on": ["artifact://current/patch"],
            "evidence": [],
            "artifacts": ["test_report"],
            "budget": {
                "cost": "medium",
                "timeout_ms": 600000
            },
            "risks": ["implementation_regression"],
            "unresolved": []
        }
    },
    {
        "id": "review_implementation",
        "target": {
            "executor": "agent",
            "capability": "technical_review"
        },
        "required": true,
        "order": "sequential",
        "depends_on": ["verify_implementation"],
        "contract": {

```

```

    "goal": "Review the implementation and verification result.",
    "constraints": ["read_only", "focus_on_correctness_and_regression"],
    "depends_on": ["artifact://current/patch", "artifact://test_report"],
    "evidence": ["artifact://test_report"],
    "artifacts": ["review_report"],
    "budget": {
      "cost": "medium",
      "timeout_ms": 600000
    },
    "risks": ["missed_runtime_regression"],
    "unresolved": []
  }
],
"on_failed": [
  {
    "id": "debug_failure",
    "target": {
      "executor": "agent",
      "capability": "debugging"
    },
    "required": true,
    "order": "sequential",
    "when": {
      "result_status": "failed",
      "retryable": true
    },
    "contract": {
      "goal": "Diagnose the failure and recommend a bounded recovery action.",
      "constraints": ["preserve_evidence", "do_not_mutate_without_assignment"],
      "depends_on": ["artifact://failure/logs"],
      "evidence": ["artifact://failure/logs"],
      "artifacts": ["debug_report"],
      "budget": {
        "cost": "medium",
        "timeout_ms": 600000
      },
      "risks": ["incorrect_recovery"],
      "unresolved": []
    }
  }
],

```

```

"on_risk_detected": [
  {
    "id": "security_review",
    "target": {
      "executor": "agent",
      "capability": "security_review"
    },
    "required": true,
    "order": "parallel",
    "when": {
      "resources": ["auth", "permission", "secret", "sandbox"],
      "side_effects": ["write"]
    },
    "contract": {
      "goal": "Review security and permission risk introduced by the change.",
      "constraints": ["read_only", "focus_on_security_regression"],
      "depends_on": ["artifact://current/patch"],
      "evidence": ["artifact://current/patch"],
      "artifacts": ["security_review_report"],
      "budget": {
        "cost": "medium",
        "timeout_ms": 600000
      },
      "risks": ["security_regression"],
      "unresolved": []
    }
  },
  {
    "limits": {
      "max_depth": 3,
      "max_parallel": 4,
      "dedupe_key": "implementation_verification_chain"
    }
  }
]
}

```

字段含义：

- **preflight**：当前 Agent 执行前可触发的准备性 Assignment，例如需求澄清、上下文研究、外部资料检索。
- **on_completed**：当当前 Agent Session 对应 assignment 成功完成后，Runtime 可以评估的编排项。
- **on_failed**：当前 assignment 失败后可触发的诊断、恢复或重试准备。
- **on_blocked**：当前 assignment 阻塞后可触发的澄清、审批或决策。

- `on_risk_detected`：Runtime 根据资源范围、side effect、权限、文件类型或 gate 识别风险后可触发的审查。
- `on_artifact_changed`：关键 Artifact 发生变化后可触发的验证、摘要、审查或索引更新。
- `on_conflict`：多个 Agent 结果冲突、gate 结果冲突或风险判断冲突后可触发的仲裁。
- `id`：编排项在当前 policy 内的稳定 id，用于 trace、dedupe 和 dependency。
- `target`：后续 executor 目标。可以指定具体 Agent，也可以指定 capability，由 Runtime routing 选择。
- `required`：该编排项是否构成 parent action 完成的必要条件。
- `order`：编排项之间的执行关系，例如 `sequential` 或 `parallel`。
- `when`：触发条件。Runtime 根据 assignment result、artifact、side effects、status 和 gate 评估。
- `depends_on`：同一 policy 内必须先完成的编排项。
- `contract`：后续 Assignment 的 Handoff Contract，包含目标、约束、依赖、证据、Artifact、预算、风险和未决问题。
- `limits`：防止自动编排过深、并发过大、重复触发或形成隐式循环的控制项。

Runtime 可以基于 `orchestration_policy` 主动创建 Assignment，但每个 Assignment 都拥有独立 authority。后续 Agent 不继承来源 Agent 的权限。

常见模式：

- 需求不清晰时，Runtime 先触发 `requirements_clarifier`。
- `cross_border_researcher` 执行前，Runtime 先触发 `amazon_product_analysis` 生成商品分析 Artifact，再把该 Artifact 作为调研输入。
- `code_developer` 完成写入后，Runtime 触发 `code_test`，再触发 `technical_reviewer`。
- `frontend_developer` 完成 UI 修改后，Runtime 触发 `accessibility_reviewer` 和 `ux_reviewer`。
- 测试失败后，Runtime 触发 `code_debugger`。
- 涉及 auth、permission 或 secret 变更时，Runtime 触发 `security_reviewer`。
- `release_runner` 完成 release preparation 后，Runtime 触发 `release_test` 或 release gate。

最小模板

```
{
  "id": "code_developer",
  "name": "Code Developer",
  "persona": "Write and verify focused code changes.",
  "description": "A coding agent for implementation tasks."
}
```

完整模板

```
{
  "id": "code_developer",
  "name": "Code Developer",
  "persona": "Write focused code changes and preserve existing behavior.",
  "description": "A development agent for implementation tasks.",
  "entry": {
    "primary": true,
    "delegable": true,
    "mentionable": true,
    "default": false,
    "hidden": false
  },
  "capability": {
    "purpose": "implementation",
    "tags": ["implementation", "debugging", "verification"],
    "cost": "medium",
    "writes": true
  },
  "model_preference": {
    "providerID": "anthropic",
    "modelID": "claude-sonnet-4-20250514"
  },
  "execution_mode": "supervision",
  "allowed_tools": ["read", "grep", "bash", "edit"],
  "denied_tools": [],
  "inherit_permissions": false,
  "permission_mode": "custom",
  "relationships": {
    "upstream": [
      {
        "id": "requirements_context",
        "target": {
          "executor": "agent",
          "capability": "requirements_clarification"
        },
        "required": false,
        "order": "sequential",
        "reason": "Implementation quality improves when unclear requirements are resolved before writing.",
        "artifacts": ["requirements_note"]
      }
    ]
  }
}
```

```

    }
  ],
  "downstream": [
    {
      "id": "implementation_verification",
      "target": {
        "executor": "agent",
        "capability": "verification"
      },
      "reason": "Written changes should be verified before review."
    }
  ]
},
"orchestration_policy": {
  "on_completed": [
    {
      "id": "verify_implementation",
      "target": {
        "executor": "agent",
        "capability": "verification"
      },
      "required": true,
      "order": "sequential",
      "contract": {
        "goal": "Verify the implementation result.",
        "constraints": ["use_changed_files", "run_relevant_checks"],
        "depends_on": ["artifact://current/patch"],
        "evidence": [],
        "artifacts": ["test_report"],
        "budget": {
          "cost": "medium",
          "timeout_ms": 600000
        },
        "risks": ["implementation_regression"],
        "unresolved": []
      }
    }
  ],
  {
    "id": "review_implementation",
    "target": {
      "executor": "agent",
      "capability": "technical_review"
    }
  }
}

```

```

    },
    "required": true,
    "order": "sequential",
    "depends_on": ["verify_implementation"],
    "contract": {
      "goal": "Review the implementation and verification result.",
      "constraints": ["read_only", "focus_on_correctness_and_regression"],
      "depends_on": ["artifact://current/patch", "artifact://test_report"],
      "evidence": ["artifact://test_report"],
      "artifacts": ["review_report"],
      "budget": {
        "cost": "medium",
        "timeout_ms": 600000
      },
      "risks": ["missed_runtime_regression"],
      "unresolved": []
    }
  },
  ],
  "limits": {
    "max_depth": 3,
    "dedupe_key": "implementation_verification_chain"
  }
}

```

SKILL.md 导入为虚拟 Agent

Open Agent Harness 可以将 `SKILL.md` 文件导入为虚拟 Agent。导入完成后，Harness 协议中的可调用对象仍是 Agent，执行实例仍是 Agent Session。

`SKILL.md` 是低摩擦编写格式，通常用于描述一类工作应该怎么做：适用场景、执行步骤、判断规则、输入要求、输出格式、质量检查和注意事项。Agent 表达可治理的执行模板，除了 prompt 材料，还包含入口规则、能力元数据、权限策略、模型偏好和运行策略。

导入层的作用，是把轻量 instruction package 归一化到 Agent registry 中，让它进入统一的 routing、delegation、permission、session creation、trace 和 UI 管理路径。

导入后的虚拟 Agent record 需要具备：

- 稳定 id
- name
- description
- prompt material

- entry flags
- capability tags
- permission profile
- source reference
- source format metadata

Runtime 后续只处理 Agent record。SKILL.md 是来源文件，虚拟 Agent 是协议对象，Agent Session 是运行实例。

支持输入

Runtime 扫描受支持 roots 中名为以下形式的文件：

```
*/SKILL.md
```

全局 root、项目 root 和插件 root 由 Runtime 配置决定。Agent 目录 root 下也可以支持同级 skill/ 和 skills/ 目录。

导入规则

每个 SKILL.md 都按 Markdown 解析，并允许可选 frontmatter。

导入后的 Agent id 来自：

1. frontmatter 中的 name，如果存在。
2. 当 name 不存在时，使用来源目录名。

id 会按 Agent 命名规范归一化为 lower_snake_case，只包含小写字母、数字和下划线。

导入后的 Agent description 来自：

1. frontmatter 中的 description，如果存在。
2. 生成的默认 description。

导入后的 prompt material 从 Markdown body 构造：

- identity、persona、职责说明等章节导入为 Agent persona。
- 步骤、规则、检查项、输出格式等章节导入为 Agent rules。
- 其他正文作为补充 prompt material 保留。

生成的 Agent metadata 使用：

```
{
  "entry": {
    "primary": false,
    "delegable": true,
    "mentionable": true,
    "default": false,
```



```

    "hidden": false
  },
  "capability": {
    "purpose": "imported_instruction",
    "tags": ["imported", "<agent-id>"],
    "cost": "medium",
    "writes": true
  },
  "inherit_permissions": false,
  "permission_mode": "lax",
  "source": {
    "format": "markdown_instruction",
    "path": "<source-dir>/SKILL.md"
  }
}

```

生成的 `meta.json`、`identity.md` 或 `rules.md` 只存在于导入结果。磁盘事实来源保持为原始 `SKILL.md` 文件。

优先级

显式 Agent 模板会覆盖相同 id 的导入结果。

这允许轻量 `SKILL.md` 逐步升级为完整 Agent 模板，并保持 invocation id 不变。

Runtime 行为

由 `SKILL.md` 导入的对象在 `/agent` 中表现为普通 Agent record，包含：

- `name`：导入后的 Agent id
- `entry`：{ "primary": false, "delegable": true, "mentionable": true, "default": false, "hidden": false }
- `capability.purpose`：imported_instruction
- `capability.tags`：["imported", id]
- `source.format`：markdown_instruction

当 Agent metadata 或配置没有隐藏该记录时，它们会出现在 session Agent 选择中。

它们默认可 mention、可 delegation，因为生成的 entry metadata 将其定位为可委托 Agent。

当 Runtime 选择该 Agent 执行工作时，运行链路与普通 Agent 一致：

Action

- > routing selects Agent
- > Runtime creates Agent Session
- > Runtime builds Context Bundle from assignment and imported prompt material

```
-> model produces protocol output
-> Runtime records Event, Artifact, Projection and Trace
```

Agent Session 的 session log 持久记录该 session 中被 Runtime 接受和引用的消息、协议输出、Action、observation、Artifact ref 和状态变化。

Context Bundle 由 Runtime 在每次模型调用前构造，包含 assignment contract、导入的 prompt material、相关 Projection、Artifact、约束、环境信息和必要的语义解释。

Agent 管理 UI

管理 API 按 Agent record 返回导入结果：

```
{
  "kind": "agent",
  "editable": false,
  "source": {
    "format": "markdown_instruction"
  }
}
```

设置 UI 可以按来源过滤：

- 全部 Agent
- 手写 Agent
- 由 `SKILL.md` 导入的 Agent

由 `SKILL.md` 导入的 Agent 在 Agent Manager 中以只读记录展示，因为它们的事实来源是原始 Markdown 文件。要定制某个导入结果，可以编辑来源文件，或创建一个相同 id 的手写 Agent 模板。

升级边界

导入层会保留主要 prompt 内容、description 和可识别的结构化章节，并生成基础 metadata。

需要精细治理时，应使用手写 Agent 模板表达更完整的字段，例如：

- 明确的 `entry`
- 细分的 `capability.purpose`
- 更准确的 `capability.tags`
- `model_preference`
- `allowed_tools` / `denied_tools`
- `permission_mode`
- `orchestration_policy`

`SKILL.md` 适合快速描述一项工作的步骤和规则；Agent 模板适合表达可治理、可路由、可观测、可授权的执行身份。

内置 Agent Entry

内置 Agent 应使用以下 entry/capability 形态。 `purpose` 是 routing hint, `writes` 和 `cost` 是 capability metadata, 不直接授予执行权限。

Agent	Entry	用途 / Capability
<code>accessibility_reviewer</code>	not primary、delegable、mentionable、not default、visible	UI 可访问性审查：语义、键盘访问、label、focus、contrast 和 assistive technology compatibility。 purpose <code>accessibility_review</code> ； writes false； cost low
<code>api_contract_reviewer</code>	not primary、delegable、mentionable、not default、visible	API contract、schema、兼容性、SDK 影响、错误语义和前后端边界审查。 purpose <code>api_contract_review</code> ； writes false； cost medium
<code>backend_developer</code>	not primary、delegable、mentionable、not default、visible	后端实现、API、数据模型、auth、permissions 和 service integration。 purpose <code>backend_implementation</code> ； writes true； cost medium
<code>code_debugger</code>	not primary、delegable、mentionable、not default、visible	复现失败、缩小问题范围、定位根因和建议下一步。 purpose <code>debugging</code> ； writes false； cost medium
<code>code_developer</code>	primary、delegable、mentionable、not default、visible	主要开发 Agent, 用于代码修改和验证。 purpose <code>implementation</code> ； writes true； cost medium
<code>code_migration_runner</code>	primary、delegable、mentionable、not default、visible	跨文件迁移、重命名、API migration 和架构迁移。 purpose <code>migration</code> ； writes true； cost high
<code>code_refactorer</code>	primary、delegable、mentionable、not default、visible	低风险、保持行为不变的重构和聚焦验证。 purpose <code>refactoring</code> ； writes true； cost medium

Agent	Entry	用途 / Capability
code_researcher	not primary、delegable、mentionable、not default、visible	文件发现、模式追踪，以及回答代码在哪里或如何实现。purpose <code>code_search</code> ；writes false；cost low
code_test	not primary、delegable、mentionable、not default、visible	运行 validation commands、解释失败并建议下一步。purpose <code>verification</code> ；writes false；cost low
data_migration_runner	primary、delegable、mentionable、not default、visible	数据迁移、schema transition、backfill、一致性检查和 rollback-aware migration plan。purpose <code>data_migration</code> ；writes true；cost high
database_developer	not primary、delegable、mentionable、not default、visible	数据库 schema、migration、query、index、transaction 和数据一致性。purpose <code>database</code> ；writes true；cost medium
dependency_maintainer	not primary、delegable、mentionable、not default、visible	依赖升级、lockfile、兼容性、breaking changes 和依赖安全风险。purpose <code>dependency_maintenance</code> ；writes true；cost medium
devops_developer	not primary、delegable、mentionable、not default、visible	CI/CD、build scripts、deployment config、本地服务、环境变量和运维设置。purpose <code>devops</code> ；writes true；cost medium
docs_maintainer	not primary、delegable、mentionable、not default、visible	维护反映当前代码和 workflow 的工程文档。purpose <code>documentation</code> ；writes true；cost low
external_researcher	not primary、delegable、mentionable、not default、visible	官方文档、远程仓库、外部库和实现示例研究。purpose <code>source_research</code> ；writes false；cost low

Agent	Entry	用途 / Capability
<code>focused_developer</code>	not primary、delegable、mentionable、not default、visible	聚焦 delegated task 的实现，不承担广义 orchestration。 purpose <code>focused_execution</code> ；writes true；cost medium
<code>frontend_developer</code>	not primary、delegable、mentionable、not default、visible	前端实现、UI 行为、样式、可访问性和浏览器验证。purpose <code>frontend_implementation</code> ；writes true；cost medium
<code>general_developer</code>	primary、delegable、mentionable、default、visible	默认通用 Agent，适合没有更明确 specialist 的普通任务。purpose <code>general</code> ；writes true；cost medium
<code>general_researcher</code>	not primary、delegable、mentionable、not default、visible	广泛研究、复杂代码库问题和并行工作单元。purpose <code>general_research</code> ；writes true；cost medium
<code>incident_responder</code>	primary、delegable、mentionable、not default、visible	事故响应、故障 triage、mitigation、verification 和后续复盘。purpose <code>incident_response</code> ；writes true；cost high
<code>multimodal_reader</code>	not primary、delegable、mentionable、not default、visible	PDF、图片、图表、diagram 和视觉文档分析。purpose <code>media_interpretation</code> ；writes false；cost low
<code>observability_developer</code>	not primary、delegable、mentionable、not default、visible	logging、metrics、tracing、audit events、health checks 和 operational diagnostics。purpose <code>observability</code> ；writes true；cost medium
<code>performance_reviewer</code>	not primary、delegable、mentionable、not default、visible	frontend、backend、runtime、database 和 build workflow 的性能审查。purpose <code>performance_review</code> ；writes false；cost medium

Agent	Entry	用途 / Capability
<code>plan_builder</code>	primary、delegable、mentionable、not default、visible	通过访谈、研究和整理创建可执行工作计划。purpose <code>plan_building</code> ; writes true; cost high
<code>plan_executor</code>	primary、delegable、mentionable、not default、visible	多步任务执行与验证协调。purpose <code>plan_execution</code> ; writes true; cost high
<code>plan_reviewer</code>	not primary、delegable、mentionable、not default、visible	审查计划是否可执行、引用是否有效、阻塞是否真实。purpose <code>plan_review</code> ; writes false; cost medium
<code>protocol_runner</code>	primary、not delegable、mentionable、not default、visible	Agent Protocol DSL 的结构化声明和协议运行观察入口。purpose <code>protocol_orchestration</code> ; writes false; cost low
<code>release_runner</code>	primary、delegable、mentionable、not default、visible	release preparation、versioning、changelog、artifact、dry run、publishing 和 post-release verification。purpose <code>release</code> ; writes true; cost high
<code>release_test</code>	not primary、delegable、mentionable、not default、visible	release artifact、version、changelog、dry run output、publish readiness 和 rollback evidence 验证。purpose <code>release_verification</code> ; writes false; cost medium
<code>requirements_clarifier</code>	not primary、delegable、mentionable、not default、visible	在 planning 或 implementation 前澄清意图、隐藏需求、歧义、风险和 planning directives。purpose <code>requirements_clarification</code> ; writes false; cost medium

Agent	Entry	用途 / Capability
security_reviewer	not primary、delegable、mentionable、not default、visible	security、permission、sandbox、secret 和 data-access 审查。purpose security_review；writes false；cost medium
session_compactor	not primary、not delegable、not mentionable、not default、hidden	长会话 continuation summary。purpose system_compaction；writes false；cost low
session_summarizer	not primary、not delegable、not mentionable、not default、hidden	隐藏系统 Agent，用于创建 session summaries。purpose system_summary；writes false；cost low
session_title_writer	not primary、not delegable、not mentionable、not default、hidden	根据首个用户 prompt 创建短 session title。purpose system_title；writes false；cost low
task_orchestrator	primary、delegable、mentionable、not default、visible	主 orchestrator：识别意图、委托 specialist、验证工作并推进交付。purpose orchestration；writes true；cost high
task_planner	primary、delegable、mentionable、not default、visible	只读规划 Agent，用于分析、设计、审查和实施计划。purpose planning_analysis；writes false；cost low
technical_reviewer	not primary、delegable、mentionable、not default、visible	架构、复杂 debugging、tradeoff 和实现后技术审查顾问。purpose technical_review；writes false；cost high
ux_reviewer	not primary、delegable、mentionable、not default、visible	UX flow、information architecture、interaction clarity、empty states 和用户摩擦审查。purpose ux_review；writes false；cost low

Agent	Entry	用途 / Capability
workflow_runner	primary、not delegable、mentionable、not default、visible	创建、保存、更新和启动 Workflow 资产；Workflow Run materialize 为统一 Action Graph 执行。purpose workflow_profile_management；writes true；cost low

协议约束

- Agent 模板的 entry、capability、permission、relationships 和 orchestration_policy 都是 Runtime 可读取的协议字段。
- 默认 Agent 选择使用 entry.primary、entry.default 和 entry.hidden，并需要可解释的 tie-breaker。
- UI picker、mention suggestions 和 delegation candidate 使用不同 entry flag。
- Relationship graph 用于解释稳定协作结构，执行仍由 Runtime 展开为 Action、Assignment 或 Handoff。
- Orchestration policy 的每个触发项都需要目标、触发条件、contract、依赖、预算、深度/并发限制和去重键。
- 每个后续 Assignment 都独立推导 authority；后续 Agent 不继承来源 Agent 的权限。

Agent Protocol DSL：模型与 Runtime 交互协议

协议目标

Agent Protocol DSL 是 Open Agent Harness 中模型与 Runtime 的交互协议。模型用结构化对象声明要做什么，Runtime 负责解析、校验、路由、执行、记录、投影、恢复和上下文构造。

协议表达模型意图，并把底层执行交给 Runtime。模型声明 semantic actions、`depends_on`、`criteria`、`failure`、`result`、context references、artifact expectation、handoff intent、recovery intent 和用户可见说明；Runtime 将这些声明归一化为 Harness Action Graph，并通过权限、预算、门禁、状态、Trace 和审计模型执行。

该协议覆盖以下场景：

- tool orchestration
- Agent delegation
- Runtime internal action
- human decision 或 approval
- context request 和 reference expansion
- structured observation
- artifact production 和 evidence collection
- handoff 或 downstream assignment
- user-visible progress
- final answer handoff

设计原则

1. 声明意图

模型声明目标、动作、依赖和结果偏好。Runtime 判断是否允许执行、由谁执行、如何执行、如何记录结果。

Model declares:

- read these sources
- run these calls in parallel
- use one result before another call
- return summary unless details are needed
- store full output as artifacts

Runtime decides:

- whether the declaration is valid
- which executor handles each call
- what authority applies
- what result is returned to the model
- what trace and projection are written

2. 协议结构保持扁平

模型更擅长生成少量顶层字段和短数组，不擅长稳定生成多层嵌套 JSON。嵌套越深，越容易出现字段遗漏、结构漂移、括号错误和语义重复。

因此，模型侧协议使用扁平 JSON shape：

- `kind`
- `message`
- `calls`
- 图级治理字段

Action Graph 通过 `calls[]` 表达 node，通过 `depends_on` 表达 dependency edge。Runtime 可以在内部把扁平 carrier 归一化为更丰富的 Action、dependency、policy、projection 和 trace records。

图级治理字段是可选字段，用于描述整个 Action Graph 的目标、约束和执行策略，例如 `title`、`goal`、`criteria`、`failure`、`budget`、`gate`、`loop`、`visibility`、`artifacts`、`handoff` 和 `context`。这些字段保持在顶层，不再包多层 envelope。

3. Tool Call 作为稳定承载方式

模型通过 Runtime 自有 toolCall endpoint `AgentProtocolOutput` 提交协议对象。这个 toolCall 是承载方式，协议语义由 `kind`、`message` 和 `calls` 定义。

使用 toolCall 承载协议有两个目的：

- 利用模型对 tool/function calling 的稳定训练能力。
- 让 Runtime 可以在同一个入口处做 schema validation、permission check、logging、projection 和 recovery。

当模型输出直接 tool request 或 delegation request 时，Runtime 可以在安全条件下恢复为同一协议 shape，再进入相同的 validation 和 execution path。

协议输出

模型到 Runtime 的输出有三种顶层 `kind`：

```
type ProtocolOutput =  
  | Act  
  | Answer  
  | Done
```

Act

act 请求 Runtime 执行一个或多个 call。

```
{
  "kind": "act",
  "message": "I will inspect the toolbar source and tests, then review the behavior.",
  "calls": [
    {
      "id": "inspect_sources",
      "type": "tool",
      "name": "glob",
      "title": "Inspect Sources",
      "args": {
        "pattern": "src/**/*toolbar*"
      },
      "result": "summary"
    },
    {
      "id": "read_toolbar",
      "type": "tool",
      "name": "read",
      "title": "Read Toolbar Code",
      "args": {
        "filePath": "src/toolbar.ts"
      },
      "depends_on": "inspect_sources",
      "result": "summary"
    },
    {
      "id": "read_tests",
      "type": "tool",
      "name": "read",
      "title": "Read Related Tests",
      "args": {
        "filePath": "src/toolbar.test.ts"
      },
      "depends_on": "inspect_sources",
      "result": "summary"
    },
    {
      "id": "review_toolbar",
      "type": "agent",
```

```

    "name": "auto",
    "title": "Review Toolbar",
    "args": {
      "description": "Review toolbar button implementation and interaction boundaries.",
      "capabilities": ["code_review", "frontend"]
    },
    "depends_on": ["read_toolbar", "read_tests"],
    "result": "structured"
  }
]
}

```

Answer

answer 在不需要 Runtime 执行更多工作时返回用户可见 Markdown。

```

{
  "kind": "answer",
  "message": "This project is a VS Code extension for visual HTML editing."
}

```

Done

done 结束当前 turn，可以包含用户可见 closing message。

```

{
  "kind": "done",
  "message": "The requested check is complete."
}

```

字段定义

字段名沿用总纲的统一字段。模型侧协议保持扁平，Runtime 归一化时补齐对象形态和默认值。

顶层字段

- **kind**：必填。取值为 **act**、**answer** 或 **done**。
- **message**：可选。用户可见 Markdown、进度说明或最终回答。**answer** 应提供 **message**。
- **calls**：**act** 必填。**answer** 和 **done** 省略。
- **title**：可选。Action Graph 的短标题，用于 Run、UI graph 和 Trace。
- **goal**：可选。Run 或 Action Graph 的目标摘要、约束和完成标准。
- **criteria**：可选 string array。整个 Action Graph 的完成标准。

- **failure**：可选 object。整个 Action Graph 的失败处理偏好，例如 `retry`、`block`、`ask_user`、`handoff` 或 `abort`。
- **budget**：可选 object。整个 Action Graph 的成本、时间、token、attempt、parallelism、缓存或外部资源约束。
- **gate**：可选 object。整个 Action Graph 的 approval、verification、review、privacy 或 release gate。
- **loop**：可选 object。有边界的重复执行语义。Runtime 根据 **max_attempts**、**until**、预算和状态证据控制执行边界。
- **visibility**：可选 object。Action Graph 结果进入 model、user、logs、trace、future_runs 的可见性偏好。
- **artifacts**：可选 array 或 object。整个 Action Graph 期望产生、读取或更新的 Artifact。
- **handoff**：可选 object。Action Graph 终态或下游交接目标、约束、依赖、证据、风险和未决问题。
- **context**：可选 object。整个 Action Graph 需要 Runtime 展开的 context refs、memory refs、artifact refs 或 projection refs。

Call 字段

- **id**：必填。稳定 call id，用于 logs、graph nodes、result references 和 dependencies。
- **type**：必填。executor class，支持 **tool**、**agent**、**runtime**、**human**、**pipeline** 或 **service**。
- **name**：必填。具体 executor target。对 **tool** 来说是 tool id；对 **agent** 来说是 Agent id 或 **auto**；对 **runtime** 来说是 Runtime 内部操作名。
- **title**：可选。短标签，用于 UI display 和 transcript heading。
- **args**：可选 object。必须匹配目标 executor 的 input schema 或 delegation contract。
- **depends_on**：可选 string 或 string array。声明该 call 必须等待哪些 call 完成。
- **result**：可选。Result return policy。可以使用 string 简写，也可以使用 object。string 允许值为 **summary**、**full**、**structured**、**on_failure**、**on_demand** 或 **adaptive**，默认 **summary**。
- **criteria**：可选 string array。声明该 call 的成功标准，用于 Runtime 构造 Action / Assignment Contract。
- **failure**：可选 object。声明失败偏好，例如 `retry`、`block`、`ask_user`、`handoff` 或 `abort`。Runtime 根据全局 policy 和 gate 解释。
- **budget**：可选 object。声明成本、时间、token、并行度或重试预算偏好。
- **visibility**：可选 object。声明结果进入 model、user、logs、trace、future_runs 的可见性偏好。
- **artifacts**：可选 array 或 object。声明期望产生、读取或更新的 Artifact，Runtime 会归一化为 Artifact Contract。
- **handoff**：可选 object。声明下游交接目标、约束、依赖、证据、风险和未决问题。
- **context**：可选 object。声明需要 Runtime 展开的 context refs、memory refs、artifact refs 或 projection refs。
- **gate**：可选 object。声明需要满足的 approval、verification、review、privacy 或 release gate。

Action Graph

`calls[]` 是模型侧 Action Graph 表达：

- 每个 `calls[]` item 是一个 graph node。
- `depends_on` 定义 node 之间的 dependency edge。
- Runtime 接受 `act` declaration 后会创建或更新持久化 Run 和 Action Graph record。
- 没有 `depends_on` 的 call 可以在 declaration 被接受后进入 ready 状态。
- 多个互不依赖的 ready call 可以并行调度。
- 具体调度受 Runtime policy、executor availability、budget、resource lock 和 permission 约束。
- Runtime 校验 missing node、duplicate id、cycle、不可满足依赖、无权限 executor 和不安全 side effect。

上面的 toolbar 示例会被 Runtime 投影为：

```
{
  "graph": {
    "nodes": [
      { "id": "inspect_sources", "operation": "inspect_sources", "executor": "tool:glob" },
      { "id": "read_toolbar", "operation": "read", "executor": "tool:read" },
      { "id": "read_tests", "operation": "read", "executor": "tool:read" },
      { "id": "review_toolbar", "operation": "review_code", "executor": "agent:auto" }
    ],
    "edges": [
      { "from": "inspect_sources", "to": "read_toolbar", "type": "depends_on" },
      { "from": "inspect_sources", "to": "read_tests", "type": "depends_on" },
      { "from": "read_toolbar", "to": "review_toolbar", "type": "depends_on" },
      { "from": "read_tests", "to": "review_toolbar", "type": "depends_on" }
    ]
  }
}
```

Action Graph 的调度基础是依赖图。Retry、loop、gate、decision、handoff、pause、resume 和恢复都归属于 Action Graph / Runtime 执行能力。

Loop 必须有边界，例如 `max_attempts`、预算、时间限制、人工 Decision 或明确 `until` 条件。Retry、loop、verification、handoff 和 post-action review 可以由模型在 Action Graph 中声明，也可以由 Agent metadata 的 Orchestration Policy 或 Runtime policy 补齐。Runtime 在执行前将这些来源归一化为统一 Action、Assignment、Gate、Event 和 Projection。

通用 Action 语义

模型侧 `calls[]` 保持扁平，Runtime 在归一化阶段补齐更完整的治理对象。

映射关系：

模型侧字段	Runtime 内部对象	语义
id	Action id	稳定引用、依赖和 Trace 关联。
type + name	Executor selector	选择 tool、Agent Session、Runtime service、human、pipeline 或 service。
args	Action input / Contract input	传给 executor 的结构化输入。
depends_on	Action Graph edge	调度、阻塞、恢复和 fan-in 的依赖边。
criteria	Success Criteria / Contract acceptance	Runtime、Agent Session、Gate 和 UI 判断完成的依据。
failure	Failure Policy	retry、block、ask_user、handoff、abort 等失败处理偏好。
budget	Budget Policy	cost、timeout、tokens、attempts、parallelism 等资源约束。
visibility	Visibility Policy	控制结果如何进入模型上下文、用户界面、日志、Trace 和未来运行。
artifacts	Artifact Contract	声明输出产物、证据、索引和引用方式。
handoff	Handoff Contract	将结果、约束、证据、风险和未决问题交给后续 executor。
context	Context Bundle request	请求 Runtime 选择、展开或摘要上下文。
gate	Gate Policy	approval、verification、review、privacy、release gate 等状态迁移条件。

这些字段都是意图声明。Runtime 根据权限、当前 Projection、Executor Registry、Adapter Policy 和安全策略决定最终执行形态。

示例：

```
{
  "id": "implement_timeout",
```

```

"type": "agent",
"name": "auto",
"title": "Implement Timeout Handling",
"args": {
  "description": "Implement typed timeout error handling for the auth refresh path.",
  "capabilities": ["implementation", "testing"]
},
"depends_on": ["inspect_auth"],
"criteria": [
  "Typed timeout error is returned by the auth refresh path.",
  "Focused tests cover success and timeout cases."
],
"failure": {
  "on_failure": "block",
  "retry": { "max_attempts": 1 }
},
"budget": {
  "timeout_ms": 900000,
  "cost": "medium"
},
"artifacts": [
  { "name": "patch", "type": "diff" },
  { "name": "test_report", "type": "verification" }
],
"handoff": {
  "target": { "type": "agent", "capability": "code_review" },
  "constraints": ["review_changed_files_only"],
  "risks": ["auth regression"],
  "unresolved": []
},
"result": "structured"
}

```

Runtime 归一化

Runtime 将模型侧协议对象归一化为内部执行表示：

- `kind: "act"` 映射到 `execute declaration`。
- 顶层 `title`、`goal`、`criteria`、`failure`、`budget`、`gate`、`loop`、`visibility`、`artifacts`、`handoff` 和 `context` 映射到 Action Graph policy、Run Contract 和 Projection。
- `calls[]` 映射到内部 Action Graph records。
- 每个 call 映射到 internal action。

- `calls[].type` 映射到 executor type。
- `calls[].name` 映射到 executor target。
- `calls[].args` 映射到 action input。
- `calls[].depends_on` 映射到 dependency edges。
- `calls[].result` 映射到 Result Policy。
- `calls[].criteria` 映射到 Success Criteria 和 Contract acceptance。
- `calls[].failure` 映射到 Failure Policy。
- `calls[].budget` 映射到 Budget Policy。
- `calls[].visibility` 映射到 Visibility Policy。
- `calls[].artifacts` 映射到 Artifact Contract。
- `calls[].handoff` 映射到 Handoff Contract。
- `calls[].context` 映射到 Context Bundle request。
- `calls[].gate` 映射到 Gate policy。
- `kind: "answer"` 映射到 response message。
- `kind: "done"` 映射到 stopped turn。

Runtime 归一化后再执行 schema validation、permission check、持久化、routing、executor invocation、event append、projection update 和 trace recording。

直接请求恢复

Runtime 可以在安全时把直接 tool request、delegation request 或文本 invocation 恢复为协议对象。

恢复条件：

- target executor 明确。
- arguments 能通过 schema 校验。
- side effects 可判断。
- permission boundary 可推导。
- 依赖关系明确。
- 执行不会绕过 approval gate。

恢复后的对象等价于显式 `AgentProtocolOutput` declaration，并进入相同的 validation、permission、logging 和 projection path。Runtime 应记录 recovery event，用于观察模型协议遵循情况。

不安全或模糊的请求会关闭执行路径，并返回 protocol error 或要求模型用协议格式重试。

Runtime 到模型：Request Transcript

Runtime-to-model request 使用模型可读 transcript。Runtime 可以在内部使用 provider messages、tool schemas 和 tool results；模型可见历史展示为简洁任务历史。

当 provider 返回 reasoning items 时，Runtime 按 provider 协议在下一次模型请求中回传这些 items，用于保持模型在 tool calling 或多步执行中的推理连续性。Reasoning items 作为 provider continuation state 传

递，模型可读 transcript 仍按本节的 Markdown 结构组织。

Transcript 包含：

- user request
- assistant protocol declaration summary
- concrete calls
- runtime observations
- artifact refs
- protocol capabilities
- next instruction

示例：

```
```markdown <turn index="1">
```

## User request

当前插件各个按钮点了都没效果，你进行一次 code review，定位问题，然后修复 </turn>

<turn index="2">

## Assistant protocol request and runtime observations

run\_id: `apr_abc123` Purpose: Inspect extension wiring Status: completed

### Call read\_extension

Tool: `read`

```
tool read <<'JSON'
{
 "filePath": "src/extension/extension.ts"
}
JSON
```

### Result for read\_extension

Status: completed Artifacts: artifact://call\_read\_extension

```
extension.ts registers the custom editor provider and command handlers.
```

</turn>

Based on all turns above, decide the next step. Strictly follow the Agent Protocol output requirements for this request. ````

Transcript 规则：

- 每个历史 turn 都应像简短 transcript 一样易读。
- Runtime turn 使用 Markdown headings 描述发生了什么。
- 每个 call 应紧邻对应 result，降低模型配对歧义。
- Result section 包含 status、artifact refs 和 result content，不重复完整 arguments。
- `run_id` 关联 logs、UI projection、hidden context 和 artifacts。
- Provider reasoning items 按 provider 协议随下一次模型请求回传。

## Runtime Observation

---

Runtime Observation 是执行结果的模型可见表示。它出现在下一次 Runtime-to-model request transcript 中。

Observation 包含：

- run id
- call ids
- titles 或 descriptions
- status
- summaries
- artifact refs
- failure 或 blocked reason
- next instruction

示例：

```
```markdown <turn index="2">
```

Assistant protocol request and runtime observations

run_id: `run_123` Purpose: Toolbar Button Review Status: completed

Call inspect_code

Executor: `tool:auto` Operation: `inspect_sources`

Result for inspect_code

Status: completed Artifacts: artifact://run_123/inspect_code

Found Toolbar.vue, ToolbarButton.vue, toolbarConfig.ts, useToolbar.ts, useEditor.ts, and related tests.

Call review_toolbar

Executor: `agent:auto` Operation: `review_code` Depends: `inspect_code`

Result for review_toolbar

Status: completed Artifacts: artifact://run_123/review_toolbar

Found two issues: undo/redo state is not wired through, and the table dropdown can be hidden by a higher layer.

</turn> ````

Runtime 也会为 logs、UI、trace export、recovery 和程序化处理存储 machine-checkable JSON records。模型可见 transcript 面向理解和下一步生成优化。

词汇与抽象边界

Declaration

模型生成的协议对象。`act` declaration 请求 Runtime 执行 calls；`answer` declaration 返回用户可见回答；`done` declaration 结束 turn。

Action Graph

由 call nodes 和 dependency edges 组成的执行声明。模型用 `calls[]` 定义 nodes，用 `depends_on` 定义 edges。

Call

模型侧的执行声明单元。Call 会被 Runtime 归一化为内部 Action。

Action

Runtime 接受后的可执行语义单元。Action 记录 operation、executor、input、dependency、authority、side effect、`result` 和 trace refs。

Operation

Action 试图完成的语义工作，回答“要做什么”。示例：`search`、`read`、`review_code`、`run_tests`、`edit`、`summarize`、`ask_user`、`expand_reference`。

Executor

执行 Action 的能力类别和目标，回答“由谁或什么执行”。示例：`tool`、`agent`、`runtime`、`human`、`pipeline`、`service`。

Operation 和 Executor 是两个维度。相同 operation 可以由不同 executor 执行，例如 `review_code` 可以由 `agent:auto`、`agent:technical_reviewer` 或 review service；相同 executor 也可以支持多个

operation, 例如 `tool` executor 可以执行 `search`、`read` 和 `run_tests`。

Target

具体 executor name 或 `auto`。示例: `read_file`、`technical_reviewer`、`ask_user`、`test_pipeline`、`auto`。

Capability

用于 routing 和 matching 的能力标签。示例: `filesystem.read`、`code_review`、`frontend`、`testing`、`approval`、`summarization`、`external.search`。

Assignment

Runtime 将 Action 绑定到 Agent Session 时创建的执行边界。Assignment 记录谁执行、授予什么 authority、提供什么 Context Bundle、期望什么结果、产生哪些 artifacts 和 trace refs。

Handoff

Assignment 或 executor 之间的结构化转移边界。Handoff 把前一个执行结果、证据、artifact refs、风险和未决问题, 与下一个目标、约束、依赖和预算一起打包。

Reference

指向 Markdown sections、先前 Action outputs、Runtime records 或 artifacts 的指针。示例: `md:review.prompt`、`action:inspect.summary`、`artifact:test_report`、`runtime://runs/run_123`、`input:user.goal`。

Result

Runtime 产生的 Action outcome。示例: `status: "completed"`、changed file list、test report summary、review findings、artifact references、failure reason。

内部 Action 字段

Runtime 将扁平 `calls[]` 归一化为内部 Action records。内部 Action 可以拥有比模型侧 call 更丰富的字段, 用于 validation、routing、UI projection 和 trace。

示例:

```
{
  "type": "action",
  "id": "review_toolbar",
  "title": "Review Toolbar",
  "description": "Review toolbar button behavior and edge cases.",
  "reason": "The user asked for per-button review.",
  "operation": "review_code",
  "executor": {
```

```

    "type": "agent",
    "target": "auto",
    "capabilities": ["code_review", "frontend"]
  },
  "depends_on": ["inspect_code"],
  "criteria": [
    "Find correctness and regression risks in toolbar behavior.",
    "Return findings with evidence and severity."
  ],
  "failure": {
    "on_failure": "block",
    "retry": { "max_attempts": 1 }
  },
  "context_refs": ["action:inspect_code.summary"],
  "artifacts": {
    "expected": [
      { "name": "review_report", "type": "review" }
    ]
  },
  "handoff": {
    "target": { "type": "agent", "capability": "implementation" },
    "constraints": ["only rework findings with evidence"],
    "depends_on": ["artifact:review_report"],
    "evidence": ["action:inspect_code.summary"],
    "risks": ["UI regression"],
    "unresolved": []
  },
  "budget": {
    "timeout_ms": 600000,
    "cost": "medium"
  },
  "gate": {
    "verification": ["evidence_present"]
  },
  "result": {
    "return_to_model": "structured",
    "store_full": true
  }
}

```

内部 Action 字段由 Runtime 生成或补全，不要求模型逐项输出。

Reference 与 Context

模型可以通过 references 表示需要的上下文材料。Runtime 负责解析 references，并在 executor invocation 或后续模型 request 中构造合适的 Context Bundle。

Reference types:

- `md:<section_id>`: 当前 Markdown content 中的 section。
- `action:<action_id>.summary`: 当前 run 中先前 Action 的 summary。
- `action:<action_id>.output`: 当前 run 中先前 Action 的 output。
- `input:user.goal`: 原始用户目标。
- `runtime://...`: 已持久化 Runtime artifact 或 record。
- `artifact:<name>`: run 产生的命名 artifact。

Runtime 直接解析 `md:` references。对 `runtime://` references，Runtime 可以自动展开，也可以允许模型声明 `expand_ref` action。

引用展开示例：

```
{
  "kind": "act",
  "message": "I need the exact evidence for the undo/redo finding.",
  "calls": [
    {
      "id": "expand_review_evidence",
      "type": "runtime",
      "name": "expand_ref",
      "args": {
        "ref": "runtime://run_123/actions/review_toolbar/output",
        "range": {
          "around_matches": true,
          "context_lines": 20
        },
        "reason": "Need exact evidence for the undo/redo finding."
      },
      "result": "excerpt"
    }
  ]
}
```

Result Policy

`result` 控制执行后返回给模型的内容。

允许值：

- `summary`：返回简洁 summary。
- `structured`：返回结构化 result fields。
- `full`：请求完整返回；Runtime 可根据预算、安全和权限限制。
- `on_failure`：只在失败时返回 detail。
- `on_demand`：返回 ref 和 summary，细节按需展开。
- `adaptive`：模型声明偏好，Runtime 在预算内选择。

Runtime 拥有最终 authority。Result policy 受 safety、privacy、permission 和 context budget 约束。

Executor Registry

协议把可执行对象视为 Runtime 注册的 executors。

Executor types：

- `tool`：确定性或有边界的系统函数。
- `agent`：由 LLM 驱动、带 reasoning 和局部自主性的 executor。
- `runtime`：Runtime 自有控制操作，例如 wait、merge、checkpoint、summarize 或 expand references。
- `human`：用户或 human operator decision。
- `pipeline`：预定义多步确定性过程。
- `service`：外部服务或集成。

暴露给 protocol-enabled Agent 的 registry information：

```
{
  "executors": [
    {
      "name": "read_file",
      "type": "tool",
      "description": "Read a file from the current workspace.",
      "capabilities": ["filesystem", "read"]
    },
    {
      "name": "technical_reviewer",
      "type": "agent",
      "description": "Reviews code changes and reports correctness, regression, and test risks.",
      "capabilities": ["code_review", "testing", "risk_analysis"],
      "can_execute": true
    },
    {
```



```
    "name": "ask_user",
    "type": "human",
    "description": "Request clarification, approval, or a decision from the user.",
    "capabilities": ["clarification", "approval"]
  }
]
```

Runtime routing 根据 call type、name、args、capabilities、task description、availability、authority、resource scope、side effect 和 cost 选择具体 executor。

Streaming

模型输出可能逐 token streaming。Runtime 只执行完整、已校验的 declaration。

规则：

- 收集完整 assistant message。
- 使用完整 `AgentProtocolOutput` toolCall arguments。
- 在 message completion 后解析并校验 arguments。
- 对文本形式的直接请求，只在目标、参数和权限边界明确时恢复。
- 执行已校验 declaration。

Runtime execution 可以 streaming progress。Progress 默认服务 UI 和 logs。模型可见 observation 只在决策点返回：

- completed
- failed
- blocked
- permission needed
- user input needed
- model decision needed

XML、JSON 与 Markdown

根据方向和用途选择格式。

Runtime 到模型

Runtime-to-model request 使用 Markdown transcript。Markdown 适合组织用户请求、执行摘要、observation、artifact refs 和下一步指令。

XML-like tags 可以作为 transcript 分组边界，例如 `<turn>`。它们用于帮助模型理解结构，语义归属于 Runtime-to-model transcript。

模型到 Runtime

模型到 Runtime 使用 `AgentProtocolOutput` `toolCall` 和扁平 JSON shape。JSON 适合 schema validation、arrays、objects、primitive data types 和 tool/function calling training patterns。

Runtime 记录

Runtime logs、UI projection、trace export、recovery 和程序化处理可以使用 machine-checkable JSON records。它们属于 Runtime 记录层，模型侧输出协议仍使用 `AgentProtocolOutput`。

模型到用户

最终回答使用普通 Markdown。

持久化

Runtime 接受 `kind: "act"` declaration 后，都会创建或更新持久化 Run 和 Action Graph。

短任务可以很快完成，但仍然通过同一套 Run、Action Graph、Action、Assignment、Event、Projection、Trace、Artifact Index、Manifest 和 Snapshot 记录执行过程。系统重启后，Runtime 根据这些记录判断哪些 Action 已完成、哪些仍在运行、哪些需要恢复、哪些应转为 `blocked`、`waiting_user` 或 `waiting_permission`。

Safety 与 Validation

Runtime 执行前校验：

- enabled Agent 才能输出可执行 protocol declaration。
- declaration 必须来自 `AgentProtocolOutput` 或可安全恢复的直接请求。
- `kind`、`message` 和 `calls` 符合 schema。
- call ids 唯一。
- dependency graph 无缺失、无环、可满足。
- executor target 可用。
- tool、file、network、service 或 external side effect 有权限。
- approval gate 被满足。
- `result` 不覆盖 safety、privacy 或 context budget。
- criteria、failure、budget、artifacts、handoff 和 gate 可以被 Runtime 解释和执行。
- Handoff 不绕过目标 executor 的 authority、budget、visibility 和 gate。

用户体验契约

Runtime 应为协议执行提供 user-facing projection：

- run title 或 purpose
- `message`

- call title
- call status
- progress
- result summary
- artifact refs
- blocker 或 failure reason
- trace export

UI 的常规展示从 message、titles、statuses 和 summaries 派生。Raw protocol record 可用于 inspection、debugging 和 advanced use。

协议边界

协议保持以下边界：

- 模型声明 intent；Runtime 拥有 execution authority。
- 模型声明 result preference；Runtime 决定实际返回粒度。
- 模型声明 dependencies；Runtime 决定 scheduling 和 concurrency。
- 模型可以请求 executor；Runtime 做最终 routing。
- 模型可请求 reference expansion；Runtime 决定展开范围。
- Harness 状态变更通过 Runtime 接受的 Action、Event 和 Projection 发生。

与 Workflow 的关系

Agent Protocol DSL 支持模型用 `calls[]` 和 `depends_on` 声明 DAG，并用 criteria、failure、budget、artifacts、handoff、gate 和 visibility 表达通用治理语义。

Workflow 是用户创建、保存或命名后的 Action Graph Profile。任务执行时模型生成的 `calls[]` 进入持久化 Action Graph；用户把这份图保存为 Workflow，或用户主动创建 Workflow 后，它成为可管理、可复用的 Workflow 资产。

Workflow Profile 可以使用更适合 UI 和用户编辑的 `nodes[]`、输入 schema、版本和可见性字段。

Runtime 接受 Workflow Profile 后，会 materialize 出新的 Action Graph，并使用与模型 `act` declaration 相同的 Action、Assignment、Event、Projection、Trace、Artifact、Gate 和恢复模型执行。

Action 与 Executor 契约

目的

本文定义 Harness 中的可执行单元。

任何可能改变执行状态的模型请求、tool call、委托 Agent 任务、Runtime 操作、人工审批、pipeline 或 service invocation，在执行前都要归一化为 `Action`。Runtime 可以接受不同的模型侧 carrier，但内部执行路径保持一致。

Action 属于持久化 Action Graph。Runtime 接受执行声明后，先创建或更新 Run、Action Graph、Action records 和 dependency edges，再根据 policy、gate 和 executor availability 调度执行。

核心规则

协议边界是 Action。Tool call 是可能的 carrier 之一。

```
model intent -> persisted Action Graph -> normalized Action -> policy checks -> executor
invocation -> result -> event/projection
```

可接受的 carrier：

- 原生 Harness DSL declaration
- `AgentProtocolOutput` 或等价的 Runtime 自有 toolCall carrier
- 从直接 tool request 安全恢复
- 从 task/delegation request 安全恢复

恢复得到的 Action 要标记 `origin.kind: "recovered"`，让日志和评测能够区分显式协议输出与恢复后的协议输入。

Action 形态

Action 字段沿用总纲的统一字段。Runtime 内部可以把模型侧简写展开为对象，但字段名保持一致。

内部 Action 字段：

```
{
  "id": "read_package",
  "type": "action",
  "operation": "read",
  "title": "Read package manifest",
  "description": "Read package.json from the current project.",
  "origin": {
    "kind": "protocol_tool_call",
```

```

    "message_id": "msg_123",
    "source_id": "call_123"
  },
  "executor": {
    "type": "tool",
    "target": "read_file",
    "capabilities": ["filesystem.read"]
  },
  "args": {
    "path": "package.json"
  },
  "resources": {
    "read": ["file:package.json"],
    "write": []
  },
  "side_effects": ["read"],
  "depends_on": [],
  "criteria": [
    "package.json is read from the current workspace.",
    "The result records package name, scripts and relevant dependencies."
  ],
  "failure": {
    "on_failure": "block",
    "retry": {
      "max_attempts": 1
    }
  },
  "permission": {
    "mode": "inherit"
  },
  "budget": {
    "timeout_ms": 120000
  },
  "result": {
    "return_to_model": "summary",
    "store_full": true
  },
  "artifacts": {
    "expected": [
      {
        "name": "package_manifest_summary",
        "type": "structured_summary",

```

```

        "visibility": "model"
    }
]
},
"handoff": {
    "target": {
        "type": "agent",
        "capability": "dependency_review"
    },
    "constraints": ["use_manifest_summary"],
    "depends_on": ["artifact://action/read_package/output"],
    "evidence": [],
    "budget": {
        "cost": "low",
        "timeout_ms": 300000
    },
    "risks": [],
    "unresolved": []
},
"visibility": {
    "model": "summary",
    "user": "summary",
    "logs": "full",
    "trace": "summary",
    "future_runs": "ref"
},
"idempotency": "safe_retry",
"cancellation": "best_effort"
}

```

稳定字段:

- `id`
- `type`
- `operation`
- `executor.type`
- `executor.target`
- `args`
- `side_effects`
- `depends_on`
- `criteria`
- `failure`

- budget
- result
- artifacts
- handoff
- visibility
- permission
- origin

Contract 语义

Action Graph Contract 描述一次 Run 或流程图的整体目标、输入、约束、依赖、预算、gate、loop、failure、Artifact 和完成标准。Action Contract 描述 Runtime 接受单个 Action 后需要维护的执行边界。

Graph-level policy、Action-level policy、Runtime policy 和 Agent metadata / Orchestration Policy 都可以提供执行策略。Runtime 在接受和调度前合并这些策略，并把最终结果记录到 Action、Assignment、Event、Projection 和 Trace。

核心字段：

字段	含义
goal / criteria	Action 要达成的目标和完成判定条件。
constraints	scope、兼容性、风格、权限、时间、成本和安全约束。
depends_on	必须先满足的 Action、Artifact、Decision 或 Projection 条件。
input / args	executor 可见的结构化输入。
artifacts	期望产生、读取、更新或引用的 Artifact。
evidence	验收和审计需要保留的证据。
budget	cost、timeout、tokens、attempts、parallelism 等资源边界。
failure	failed、blocked、retry、ask_user、handoff、abort 等处理语义。
handoff	后续 executor 接力时需要携带的目标、约束、证据、风险和未决问题。
visibility	结果进入 model、user、logs、trace 和 future runs 的规则。
loop	有边界的重复执行语义，必须有 attempt、预算、时间、条件或 Decision 边界。

Action Contract 是 Runtime、Executor、UI 和后续 Agent Session 共同读取的治理对象。模型可以声明其中一部分，Runtime 根据上下文补齐可执行边界。

Artifact 语义

Artifact 是 Action 或 Executor 产生的可引用产物。Runtime 使用 Artifact ref 把大型输出、证据和中间结果从模型上下文中分离出来。

Artifact record 字段：

```
{
  "id": "artifact_read_package_output",
  "type": "structured_summary",
  "name": "package_manifest_summary",
  "producer": "action:read_package",
  "scope": "run",
  "uri": "artifact://action/read_package/output",
  "summary": "package.json scripts and dependencies summary.",
  "status": "available",
  "visibility": {
    "model": "summary",
    "user": "summary",
    "logs": "full",
    "trace": "summary",
    "future_runs": "ref"
  },
  "evidence": ["event:action.output_stored"]
}
```

Artifact Contract 定义期望产物，Artifact record 记录实际产物。Runtime 在 Action result、Event、Projection 和 Trace 中引用同一个 Artifact id。

Gate 与 Budget 语义

Gate 是状态迁移或执行前的强制检查。常见 Gate 包括 schema、authority、scope、dependency、approval、verification、review、privacy、release 和 budget。

Budget Policy 约束资源使用：

- `timeout_ms`：执行时间上限。
- `cost`：成本等级或预算分类。
- `tokens`：模型输入、输出和 reasoning token 预算。
- `max_attempts`：重试次数。
- `parallelism`：可同时运行的 executor 数。

- `cache`：缓存读取、写入和复用策略。

Runtime 在执行前校验 Gate，在执行中更新消耗，在状态投影中暴露剩余预算和阻塞原因。

Executor 类型

Runtime 支持的 executor class：

Type	含义	执行语义
<code>tool</code>	有边界的 tool 或 MCP tool	按 tool schema 执行，受 resource、side effect、permission 和 <code>result</code> 约束。
<code>agent</code>	委托 LLM session	Runtime 创建 Assignment 和 Agent Session，并记录 child-session trace。
<code>runtime</code>	Harness 自有操作，例如 summarize、checkpoint、merge、wait	Runtime service 在同一状态事务模型中执行。
<code>human</code>	用户/Owner 澄清、审批或决策	Runtime 创建 pending decision，并由 UI/API 收集结果。
<code>pipeline</code>	预定义确定性多步过程	Pipeline 被视为一个 executor，内部步骤写入 trace。
<code>service</code>	外部集成	Service invocation 受 network、secret、approval、budget 和 artifact policy 约束。

执行环境契约

Executor 执行绑定到 Runtime 创建的环境。环境契约定义 executor 在哪里运行、能看到什么状态、能改变什么状态，以及 Runtime 如何在失败或重启后恢复它。

核心环境对象：

- **Sandbox**：executor 的隔离执行边界，覆盖进程、文件系统、网络、secret、外部服务和副作用。
- **Workspace**：executor 可见的工作区域，例如项目目录、worktree、run-scoped 目录、临时目录或挂载的 artifact set。
- **Manifest**：Runtime 提供的环境声明，包含 cwd、读写范围、可用工具、env refs、secret refs、网络策略、artifact 输出策略、snapshot refs 和 timeout。
- **Snapshot**：某个时间点的 workspace、materialized state、artifact index 或 executor state 检查点，用于对比、审计、rollback 判断和恢复。

- **Rehydration**: Runtime 根据 Manifest、Snapshot、Event、Projection 和 Artifact refs 重建 executor 环境或 Agent Session context 的过程。

执行行为：

- Runtime 根据 Action resources、Assignment authority、run policy 和 gate result 创建环境。
- Executor 只接收 Manifest 声明的环境能力。
- Executor 输出被存储为 result summary、logs、artifact refs 和 trace entries。
- Runtime 在 Event 和 Trace 中记录 environment refs，让执行可以被审计和恢复。
- Rehydration 会在继续 suspended、failed 或 long-running Action 前重建环境。

Manifest 字段：

```
{
  "id": "manifest_read_package",
  "action_id": "read_package",
  "sandbox": {
    "mode": "workspace_read"
  },
  "workspace": {
    "cwd": ".",
    "read": ["package.json"],
    "write": []
  },
  "tools": ["read_file"],
  "network": "disabled",
  "secrets": [],
  "artifacts": {
    "output": "artifact://action/read_package/output"
  },
  "snapshots": [],
  "timeout_ms": 120000
}
```

ToolCall 恢复

当模型输出直接 tool request 时，Runtime 只有在以下条件成立时才可以将其恢复为 Action：

- 该 tool 能映射到唯一 executor target
- 参数能通过对应 executor schema 校验
- side effects 可以分类
- permission policy 已知
- `result` 可以安全默认

- 执行不会绕过已禁用的 Agent/tool policy

不安全的恢复应关闭执行路径。

示例：

- `read_file({ path: "package.json" })` 可以恢复为 `operation: "read"`。
- `bash({ command: "rm -rf dist" })` 需要显式 side effect 分类和审批。
- `task({ description: "fix it" })` 过于模糊，除非 target agent、scope 和 expected result 都能安全推导。

生命周期

Action 生命周期事件：

```
action.graph_persisted
action.accepted
action.graph_ready
action.blocked
action.started
action.executor_selected
action.permission_requested
action.permission_resolved
action.retry_scheduled
action.loop_attempt_started
action.loop_attempt_completed
action.output_stored
action.completed
action.partially_completed
action.failed
action.cancelled
```

每个终态 Action result 应包含：

- `status`
- `summary`
- `artifact_refs`
- `logs_ref`
- 失败或阻塞时的 `error`
- `criteria` 的满足情况
- `failure` 的处理结果
- `handoff` 或 pending decision
- 回放给模型内容的 `visibility` 摘要

协议能力边界

Action / Executor 契约覆盖以下能力：

- 模型声明和 toolCall carrier 归一化为 Action。
- Action Graph dependency、criteria、failure、budget 和 result。
- Tool、Agent Session、Runtime service、human、pipeline 和 service executor。
- Sandbox、Workspace、Manifest、Snapshot 和 Rehydration。
- Artifact、Trace、Projection 和 Event 的统一引用。
- Handoff 和 downstream Assignment。
- Runtime 对 side effect、permission、privacy、approval 和 budget 的 gate enforcement。
- Action Graph 持久化、pause / resume、retry、bounded loop、Decision 和系统重启后的恢复。

路由与委托策略

目的

本文定义 Harness 如何选择 executor，以及如何委托工作。

Routing 将归一化后的 `Action` 转换为可执行的 `Assignment` 或直接 executor invocation。Delegation 是一种 routing 结果，其 executor 是 Agent Session。

输入

Runtime routing 使用：

- action `operation`
- 请求的 `executor.type`
- 请求的 `executor.target`
- 所需 capability
- resource 读写范围
- side effects
- Agent `entry`
- Agent `capability`
- Agent `relationships`
- Agent `orchestration_policy`
- 生效 permission policy
- 可用性、成本和模型偏好
- 当前 run state 和 dependency state

多 Agent 适用条件

当任务形态能从分离的执行上下文中获益时，Runtime 应将工作路由给多个 Agent Session。

适用条件：

- **任务可分解**：工作可以拆成相对独立的子目标，且每个子目标都有清晰输入和验收标准。
- **上下文可隔离**：每个子任务可以在独立 Context Bundle 中运行，而不继承完整会话历史。
- **成本收益匹配**：并行 session 带来的质量、覆盖率、速度或风险降低，足以覆盖额外模型、工具和协调成本。
- **结果可聚合**：子任务结果可以通过 artifact、summary、evidence、status 和 unresolved issues 合并回 parent run。
- **治理边界清晰**：每个子任务都能获得清晰的 authority、contract、trace 和 handoff boundary。

高度耦合且依赖共享可变上下文的工作，更适合先由一个 Agent Session 完成，直到 Runtime 能定义可靠的 contract boundary。

候选过滤

候选过滤顺序：

1. 从 `executor.type` 选择 executor class。
2. 如果 `target` 是具体对象，加载该 executor，缺失则拒绝。
3. 如果 `target` 是 `auto`，根据 registry metadata 构建候选。
4. 拒绝 hidden 或 disabled Agent，除非 Runtime 拥有明确 system authority。
5. 对 Agent delegation，要求 `entry.delegable === true`。
6. 匹配 capability purpose 和 tags。
7. 拒绝 permission profile 无法满足 Action side effects 的候选。
8. 应用成本、可用性和模型约束。
9. 选择最少意外的候选，并记录该决策。

模型可以建议 executor。最终选择由 Runtime 控制。

Assignment 绑定

当 Action 委托给 Agent 时，Runtime 创建 assignment：

```
{
  "id": "assign_review_changes",
  "action_id": "review_changes",
  "agent_id": "technical_reviewer",
  "capabilities": ["code_review", "testing"],
  "authority": {
    "read": ["repo://current"],
    "write": [],
    "approve": ["task.review"]
  },
  "contract": {
    "goal": "Review the current patch for correctness and regression risk.",
    "input": "context_bundle",
    "constraints": ["read_only", "focus_on_changed_files"],
    "depends_on": ["artifact://diff/current"],
    "evidence": ["finding_refs", "test_refs"],
    "artifacts": ["review_report"],
    "budget": {
      "cost": "medium",
      "timeout_ms": 600000
    }
  }
}
```

```
  },
  "risks": ["missed_runtime_regression"],
  "unresolved": [],
  "output": "evaluation_result"
}
}
```

模板级 metadata 本身不授予 authority。Runtime 根据 Action policy、run policy、用户审批和 gate requirements 推导 assignment authority。

Child Session Trace

Agent delegation 必须产生可检查的 child records:

- parent run id
- parent action id
- child session id
- assigned agent id
- 生效 capability match
- 生效 authority
- contract summary
- input context refs
- result summary
- artifact refs
- unresolved issues
- failure/block reason

失败的 child work 不能从 parent action 中静默丢弃。Parent action 应根据 `failure` 进入 `failed`、`blocked` 或 `partial`。

Handoff 契约

从一个 Agent 到另一个 Agent 的 handoff 通过 Runtime 表示:

```
Agent A -> action result / command -> Runtime -> assignment -> Agent B
```

Agent A 可以提出 next Action 或 handoff request。Runtime 校验该请求、创建 Assignment，并将工作分发给 Agent B。

Handoff 应保留 contract boundary，避免把自由文本 transcript continuation 当作交接机制。

Handoff 字段:

```
{
  "goal": "Rework the failing timeout handling after review.",
  "source_session": "session_code_developer",
  "target": {
    "executor": "agent",
    "capability": "implementation"
  },
  "constraints": ["keep_public_api", "touch_auth_module_only"],
  "depends_on": ["artifact://review/findings"],
  "evidence": ["artifact://test/logs/auth_timeout"],
  "artifacts": ["artifact://patch/current"],
  "budget": {
    "cost": "medium",
    "timeout_ms": 900000
  },
  "risks": ["retry_loop_regression"],
  "unresolved": ["confirm whether timeout should be configurable"]
}
```

Runtime 使用该结构创建下一个 Assignment、构造目标 Context Bundle，并保持 session 之间的 trace continuity。

`05-handoff-protocol.md` 定义 Handoff 的生成细节：source Agent self-report、raw records、structured trace、Handoff Source Bundle、handoff writer、canonical record 和目标 Markdown Context Bundle。本文只定义 routing 和 delegation 如何消费 Handoff Contract。

Runtime Orchestration Policy 展开

Agent 模板可以声明 `orchestration_policy`。Runtime 在 Agent Session 执行前、执行中和进入终态后评估该策略，并在满足条件时创建额外 Action / Assignment。

该机制用于表达 Runtime 主动编排 Agent 的常见模式，例如：

```
user request accepted
-> Runtime evaluates orchestration_policy.preflight
-> requirements_clarifier assignment
-> code_developer assignment
-> Runtime evaluates orchestration_policy.on_completed
-> code_test assignment
-> technical_reviewer assignment
-> parent action completed
```

Runtime 展开 `orchestration_policy` 时遵循以下规则：

1. 读取相关 Agent 模板中的 `orchestration_policy`，并根据触发点选择 `preflight`、`on_completed`、`on_failed`、`on_blocked`、`on_risk_detected`、`on_artifact_changed` 或 `on_conflict`。
2. 结合 user request、assignment result、Artifact、side effects、resource scope、run state、gate result 和已有 trace 评估 `when` 条件。
3. 对每个命中的编排项，构造标准 Contract；需要交给后续 Agent 或 human 时，构造 Handoff Contract。
4. 将编排项转换为新的 Action 或 Assignment，并进入正常 routing 流程。
5. 对目标 Agent 使用 `entry.delegable`、`capability`、permission profile、availability 和 budget 做候选过滤。
6. 可读取 Agent `relationships` 解释上游、下游、替代和冲突关系，但不能把 relationship 当成已授权调用。
7. 为后续 Assignment 独立推导 authority；后续 Agent 不继承来源 Agent 的权限。
8. 追加 orchestration / handoff 相关 Event，更新 Projection 和 Trace。
9. 根据 `required` 判断 parent action 是否必须等待该编排项完成。
10. 根据 `limits.max_depth`、`limits.max_parallel`、`limits.dedupe_key` 和历史 trace 防止重复触发、隐式循环和无限编排。

编排项的 `contract` 必须保留完整边界：

- `goal`
- `constraints`
- `depends_on`
- `evidence`
- `artifacts`
- `budget`
- `risks`
- `unresolved`

展开后的内部 Action 形态：

```
{
  "id": "orchestrate_verify_implementation",
  "type": "action",
  "operation": "orchestrate",
  "origin": {
    "kind": "agent_orchestration_policy",
    "source_assignment_id": "assign_code_developer",
    "trigger": "on_completed",
    "policy_id": "verify_implementation"
  },
  "executor": {
    "type": "agent",
```

```

    "target": "auto",
    "capabilities": ["verification"]
  },
  "depends_on": ["assign_code_developer"],
  "result": {
    "return_to_model": "summary",
    "store_full": true
  }
}

```

展开后的 Assignment contract:

```

{
  "id": "assign_code_test",
  "action_id": "orchestrate_verify_implementation",
  "agent_id": "code_test",
  "authority": {
    "read": ["repo://current", "artifact://current/patch"],
    "write": [],
    "approve": []
  },
  "contract": {
    "goal": "Verify the implementation result.",
    "constraints": ["use_changed_files", "run_relevant_checks"],
    "depends_on": ["artifact://current/patch"],
    "evidence": [],
    "artifacts": ["test_report"],
    "budget": {
      "cost": "medium",
      "timeout_ms": 600000
    },
    "risks": ["implementation_regression"],
    "unresolved": []
  }
}

```

自动编排失败时，Runtime 根据 `required` 和 `failure` 决定 parent action 状态：

- `required: true` 的编排项失败会使 parent action 进入 `blocked`、`failed` 或 `partial`。
- `required: false` 的编排项失败会记录 trace 和 warning，但不必阻塞 parent action 完成。
- 目标 Agent 不存在、不可 delegation、权限不满足或预算不足时，Runtime 应记录 orchestration blocked reason。

当编排链需要复杂条件分支、循环、并行 fan-out/fan-in 或跨 session 长时间恢复时，Runtime 通过持久化 Action Graph、Graph-level policy、Agent metadata / Orchestration Policy、Decision、Gate 和 Handoff 表达这些语义。

用户将这类 Action Graph 保存为 Workflow，或主动创建 Workflow 时，它成为 Workflow 资产；执行能力仍来自同一套 Action Graph 和 Runtime 状态模型。

Delegation Request 恢复

如果模型输出直接 task/delegation request，Runtime 只有在以下条件成立时才可以将其恢复为 Agent Action：

- target Agent 具体或可安全解析
- description 足够具体，可以形成 assignment
- scope 和 expected result 已知
- permissions 可以 enforcement

否则 Runtime 应返回协议违规，并要求结构化 Action。

协议能力

Routing 与 Delegation 策略覆盖以下能力：

- 使用 `entry`、`capability`、authority、budget 和 availability 做 `target: "auto"` Agent 选择。
- 为 Agent delegation 创建 Assignment、Contract 和 Agent Session。
- 记录 child session trace links、artifact refs、result summary、unresolved issues 和 failure reason。
- 对 hidden、disabled、non-delegable 或 permission-mismatched Agent 执行 gate enforcement。
- 将 Handoff Contract 交给后续 Agent Session 或 human owner。
- 将复杂条件分支、循环、fan-out/fan-in 和跨 session 长时间恢复表达为持久化 Action Graph、Decision、Gate、Handoff 和 Runtime Orchestration Policy。

Handoff 协议

状态: Draft

日期: 2026-06-02

相关文档:

- `02-model-runtime-protocol.md`
- `04-routing-and-delegation-policy.md`
- `06-state-event-projection-model.md`
- `07-context-memory-visibility-policy.md`
- `09-ui-console-and-agent-management.md`

目标

Handoff 协议定义一个 Agent Session 如何把可继续使用的工作状态交给另一个 Agent Session、human owner 或 Action Graph node。

它处理的是一个执行问题: 下一个 executor 要看懂上一个会话做了什么、留下了什么、哪里有风险、下一步该从哪里接, 同时不默认读取完整且嘈杂的 transcript。

完整 transcript 里可能包含重复工具输出、失败尝试、用户中途修正、过期假设和无关日志。短总结又可能漏掉关键现场。Handoff 因此使用四类输入:

- `raw records`: 保存现场原文。
- `structured trace`: 建立过程索引。
- `source Agent self-report`: 提供现场语义线索。
- `handoff_writer output`: 压缩成下一个会话能读的交接草稿。

最终 Handoff record 由 Runtime 生成和持有。模型可以起草、压缩和解释; Runtime 负责记录原始材料、校验引用、执行 visibility/redaction、写入 Event, 并为目标会话构造 Context Bundle。

核心流程

source Agent Session 执行任务

- > Runtime 保存 raw records 和 structured trace
- > Runtime 判断是否到达 handoff boundary
- > Runtime 在需要时请求 source Agent 生成轻量 self-report
- > Runtime 构造 Handoff Source Bundle
- > handoff_writer 将 source bundle 压缩为 handoff draft
- > Runtime 归一化、校验并保存 canonical Handoff record
- > 目标 Context Bundle 收到由 Handoff record 渲染出的 Markdown

handoff 的触发权在 Runtime。source Agent 可以提出 handoff intent，但 intent 只是输入，不等于正式交接。Runtime 需要根据 run state、assignment state、policy、用户操作和上下文预算决定是否创建 handoff。

Runtime 在这些场景计划 handoff：

- **Assignment 终态**：Assignment `completed`、`partial`、`blocked`、`failed`，或执行被取消但已有可用现场。
- **下游依赖存在**：当前 Assignment 有 `next`、review、test、merge、follow-up、fan-in 聚合或其他 depends。
- **模型提出 intent**：source Agent 在协议输出里声明需要交给 reviewer、tester、human owner 或另一个 capability。
- **Orchestration Policy 命中**：例如 implementation 结束后进入 code review，review 结束后进入 test audit。
- **用户或 UI 手动触发**：用户指定“交给下一个 Agent”、“让 human 看一下”或从 Console 创建 continuation。
- **Runtime 限制触发**：context budget 接近耗尽、预算不足、权限不足、工具不可用或当前 executor 无法继续。
- **final continuation**：final answer 后需要为 future run 留下可恢复的工作状态。

Runtime 的计划结果应包含：

```
{
  "type": "handoff.plan",
  "trigger": true,
  "reasons": ["assignment_terminal", "downstream_next"],
  "request_self_report": true,
  "status": "completed"
}
```

source Agent 可以在协议输出里提出 handoff 意图。Runtime 决定是否接受、补全、降级或用已保存证据重新生成。

设计规则

1. Handoff 以 Runtime record 保存；普通聊天消息只作为 raw evidence。
2. raw records 与 handoff summary 分开保存。
3. structured trace 只负责索引 raw records，不替代 raw records。
4. source Agent self-report 由 Runtime 在边界处请求；它是语义线索，不能直接当作事实。
5. 普通 Agent prompt 不承担完整 Handoff schema；复杂格式由 Runtime 和 handoff_writer 处理。
6. handoff_writer 读取 Source Bundle，并在需要时展开 raw refs。
7. 目标 Agent 默认接收 Markdown，不直接接收 canonical JSON。
8. 关键 claim 应尽量带 trace、artifact 或 raw ref。

9. Runtime 在交给目标 Agent 前执行 visibility、redaction、authority 和 budget 检查。

角色分工

Source Agent

source Agent 执行任务。它在 terminal state 或 handoff boundary 收到 Runtime 的轻量提示后，留下一个短 self-report。

self-report 应保持扁平：

```
Status: partial
Goal: Fix auth timeout handling.

Done:
- Added timeout error path.
- Updated focused tests.

Artifacts:
- artifact://patch/current
- artifact://test/auth-timeout

Unresolved:
- Broader auth test suite still needs to run.

Risks:
- Only focused tests were run.

Next:
- Run broader auth tests.
- Review timeout behavior in refresh retry path.
```

self-report 的价值在于暴露 source Agent 认为重要的信息，例如取舍、风险、未验证项和推荐下一步。canonical handoff 由 Runtime 另行生成。

Runtime 不应每轮都要求 self-report。它只在边界处请求：

- Assignment 即将结束。
- source Agent 声明 blocked、partial 或 failed。
- Runtime 准备创建下游 session。
- 用户要求交接。
- context 或预算即将中断，需要保留现场。

如果 source Agent 未生成 self-report, Runtime 仍然继续 handoff 流程, 用 structured trace 和 raw refs 生成 minimal handoff, 并标记证据不完整。

Runtime

Runtime 维护执行边界:

- 保存 raw records。
- 追加 Event records。
- 维护 structured trace。
- 索引 Artifact records。
- 判断是否触发 handoff plan。
- 在边界处请求 source Agent 生成轻量 self-report。
- 构造 Handoff Source Bundle。
- 在需要时调用 handoff_writer。
- 归一化并校验 Handoff records。
- 将 Handoff record 渲染为目标 Markdown。
- 写入 `handoff.self_report_requested`、`handoff.created`、`handoff.updated` 或 `handoff.rejected` events。

Runtime 不需要理解每一句领域语义。它需要保存足够材料和引用, 让后续 reducer、Agent 或 human owner 可以复查。

Handoff Writer

`handoff_writer` 是 hidden system Agent 或 reducer。它不获得执行用户任务的 authority, 只负责整理 Runtime 提供的证据。

它接收:

- source Agent self-report
- structured trace timeline
- 相关 artifact summaries
- 关键 raw transcript excerpts
- full transcript 和 tool logs 的 raw refs
- assignment contract
- user goal
- decisions、blockers 和 unresolved questions

它输出 summary、facts、artifacts、unresolved、risks 和 next steps。Runtime 校验 draft 后再保存。

Target Agent

target Agent 收到带 Handoff Markdown 的 Context Bundle。它可以通过正常的 context request 或 tool execution path 请求展开 refs。

target Agent 不继承 source Agent 的 authority。新的 Assignment 需要独立 routing、permission、budget 和 gate 检查。

Raw Records

raw records 保存经过 Runtime 的原始材料：

- user messages
- model-visible context snapshots
- model outputs
- protocol declarations
- tool call arguments
- tool results
- command stdout 和 stderr
- patches、diffs 和 changed file snapshots
- generated artifacts
- permission requests 和 decisions
- UI 或 human owner decisions

raw records 可以存为 artifact、log 或 session archive segment。Handoff record 引用它们，不默认内联完整内容。

示例：

```
artifact://raw/session/session_dev_01/full
artifact://raw/model/output_006
artifact://raw/tool/test_stdout_010
artifact://patch/fix_undo_redo_state
```

Structured Trace

structured trace 是 Runtime 对 raw material 建立的结构化过程索引。它记录顺序、归属、状态和引用。

structured trace 只提供过程索引。无损依赖 raw records。trace 的职责是让 raw records 可以定位、聚合、路由和审计。

Runtime 在这些边界写入 trace：

- assignment started / completed
- model context built
- model output received
- protocol declaration accepted / recovered
- Action accepted / started / completed / failed / blocked
- executor selected

- tool call requested
- tool result returned
- artifact declared / indexed
- patch applied
- test command completed
- permission requested / resolved
- decision requested / applied
- handoff created / updated / rejected

Trace entry 形态：

```
{
  "seq": 10,
  "type": "tool.result",
  "run_id": "run_123",
  "session_id": "session_dev_01",
  "assignment_id": "assign_fix_toolbar",
  "action_id": "rerun_tests",
  "actor": {
    "type": "tool",
    "id": "bash"
  },
  "status": "completed",
  "summary": "Focused undo/redo state tests passed.",
  "refs": {
    "raw": "artifact://raw/tool/test_stdout_010",
    "artifact": "artifact://test/toolbar_passed_001"
  },
  "time": "2026-06-02T10:20:00Z"
}
```

Trace entry 的 identity、order、type、status、actor 和 refs 来自 Runtime instrumentation。summary 可以来自工具、executor result 或 reducer，但需要保留 raw ref。

Handoff Source Bundle

Runtime 在调用 handoff_writer 前构造 Handoff Source Bundle。

Source Bundle 作为 reducer 输入；最终 handoff 由 Runtime 另行生成。

```
{
  "type": "handoff.source",
  "version": "1",
```

```

    "id": "source_handoff_fix_toolbar",
    "source": {
      "run_id": "run_123",
      "session_id": "session_dev_01",
      "assignment_id": "assign_fix_toolbar",
      "agent_id": "code_developer"
    },
    "status": "completed",
    "goal": "Fix toolbar undo/redo buttons and run focused tests.",
    "contract": {
      "constraints": ["touch toolbar/editor state files only"],
      "required_artifacts": ["patch", "test_report"]
    },
    "self_report": {
      "summary": "Undo/redo state refresh was fixed and focused tests passed.",
      "risks": ["Full package suite was not run"]
    },
    "timeline": [
      {
        "seq": 5,
        "type": "tool.result",
        "summary": "Toolbar button dispatches command ids, but undo/redo disabled state comes from stale editor state.",
        "refs": ["artifact://source/toolbar_relevant_files"]
      },
      {
        "seq": 7,
        "type": "file.patch_applied",
        "summary": "Updated useEditorState.ts to refresh canUndo/canRedo after editor transactions.",
        "refs": ["artifact://patch/fix_undo_redo_state"]
      },
      {
        "seq": 10,
        "type": "tool.result",
        "summary": "Focused undo/redo state tests passed.",
        "refs": ["artifact://test/toolbar_passed_001"]
      }
    ],
    "artifacts": [
      {
        "ref": "artifact://patch/fix_undo_redo_state",

```

```
    "type": "patch",
    "summary": "Patch for editor state refresh."
  },
  {
    "ref": "artifact://test/toolbar_passed_001",
    "type": "test_report",
    "summary": "Focused undo/redo state tests passed."
  }
],
"decisions": [],
"unresolved": ["Full package test suite was not run."],
"raw_refs": [
  "artifact://raw/session/session_dev_01/full",
  "artifact://raw/tool/test_stdout_010"
]
}
```

Source Bundle 分三层：

层级	用途	示例
Level 1	必读上下文	goal、status、contract、 timeline、artifacts、 unresolved、self-report
Level 2	关键原文片段	final response、failed command excerpt、user correction、 important model note
Level 3	原始材料引用	full transcript、full tool logs、 raw artifacts

短会话可以内联更多 raw excerpts。长会话应优先提供 raw refs 和少量关键 excerpts。

Handoff Writer 输出

handoff_writer 输出应保持扁平，便于校验和渲染。

```
{
  "status": "completed",
  "summary": "Undo/redo toolbar state fix was implemented and focused tests passed.",
  "facts": [
    {
      "text": "Undo/redo state now refreshes after editor transaction updates.",
```

```

    "refs": ["artifact://patch/fix_undo_redo_state"]
  },
  {
    "text": "Focused undo/redo state tests passed.",
    "refs": ["artifact://test/toolbar_passed_001"]
  }
],
"artifacts": [
  "artifact://patch/fix_undo_redo_state",
  "artifact://test/toolbar_passed_001"
],
"decisions": [],
"risks": ["Full package test suite was not run."],
"unresolved": ["Full package test suite was not run."],
"next": [
  "Run broader package tests if preparing for merge.",
  "Review whether other toolbar buttons depend on the same transaction-state refresh path."
],
"raw_refs": ["artifact://raw/session/session_dev_01/full"]
}

```

没有 ref 的 claim 不能进入 evidenced facts。Runtime 可以把它降级为 `note` 或 `assumption`。

Canonical Handoff Record

Runtime 将 writer output 归一化为 canonical Handoff record。

```

{
  "type": "handoff",
  "version": "1",
  "id": "handoff_fix_toolbar_to_review",
  "source": {
    "run_id": "run_123",
    "session_id": "session_dev_01",
    "assignment_id": "assign_fix_toolbar",
    "agent_id": "code_developer"
  },
  "target": {
    "executor": "agent",
    "capability": "code_review"
  },
  "status": "completed",

```

```

"goal": "Review the toolbar undo/redo state fix.",
"summary": "Undo/redo toolbar state fix was implemented and focused tests passed.",
"facts": [
  {
    "text": "Undo/redo state refreshes after editor transaction updates.",
    "refs": ["artifact://patch/fix_undo_redo_state"],
    "confidence": "evidenced"
  },
  {
    "text": "Focused undo/redo state tests passed.",
    "refs": ["artifact://test/toolbar_passed_001"],
    "confidence": "evidenced"
  }
],
"artifacts": [
  {
    "ref": "artifact://patch/fix_undo_redo_state",
    "type": "patch",
    "status": "available",
    "summary": "Patch for editor state refresh."
  },
  {
    "ref": "artifact://test/toolbar_passed_001",
    "type": "test_report",
    "status": "available",
    "summary": "Focused undo/redo state tests passed."
  }
],
"decisions": [],
"constraints": ["Review changed toolbar/editor state files first."],
"risks": ["Full package test suite was not run."],
"unresolved": ["Full package test suite was not run."],
"next": [
  {
    "goal": "Run broader package tests if preparing for merge.",
    "depends_on": ["artifact://patch/fix_undo_redo_state"]
  }
],
"raw_refs": ["artifact://raw/session/session_dev_01/full"],
"visibility": {
  "model": "summary",
  "user": "summary",

```

```
"logs": "full",
"trace": "summary",
"future_runs": "ref"
},
"created_by": "runtime"
}
```

Runtime 校验项:

- referenced artifacts 存在, 且 target 可见。
- trace refs 属于 source run 或允许的 upstream run。
- 无 refs 的 claim 被标为 notes 或 assumptions。
- target executor 可用, 并且独立授权。
- redaction policy 已在 target context rendering 前应用。
- unresolved items 没有被无证据删除。
- raw refs 被保留, 用于 audit 和 on-demand expansion。

目标 Markdown 渲染

target Agent 默认收到 Markdown, 不直接读 canonical JSON。

```
## Handoff

Source: `code_developer`
Status: `completed`

### Goal

Review the toolbar undo/redo state fix.

### What Was Done

- Undo/redo state now refreshes after editor transaction updates.
  Ref: `artifact://patch/fix_undo_redo_state`

- Focused undo/redo state tests passed.
  Ref: `artifact://test/toolbar_passed_001`

### Available Artifacts

- `artifact://patch/fix_undo_redo_state`
  Patch for editor state refresh.
```

```
- `artifact://test/toolbar_passed_001`  
  Focused undo/redo state test report.  
  
### Remaining Risk  
  
- Full package test suite was not run.  
  
### Suggested Next Steps  
  
1. Run broader package tests if preparing for merge.  
2. Review whether other toolbar buttons depend on the same transaction-state refresh path.  
  
### Raw Evidence  
  
- Full source session: `artifact://raw/session/session_dev_01/full`
```

Markdown 是 canonical Handoff record 的投影，不作为事实来源。

噪音控制

Runtime 在 handoff 进入目标上下文前应控制噪音：

- 合并 action signature 相同的重复 tool calls。
- 只保留能解释 decision、risk 或 blocker 的 failed attempts。
- 将被后续证据推翻的 hypotheses 标为 superseded。
- 保留用户修正和后续决策。
- 按文件、命令和 artifact 分组。
- full stdout 和 stderr 保留在 raw refs 中。
- validation artifacts 保留精确 command string。
- 不复制无关 child session transcripts。

如果 self-report 与 structured trace 冲突，handoff 应暴露冲突：

```
### Conflict  
  
- Source self-report says focused tests passed.  
- Structured trace has no passing test artifact.  
- Treat test status as unresolved until a test artifact is produced.
```

信息保全

Handoff 不能保证每个语义细节都被摘要捕捉。协议要让遗漏更容易发现和恢复。

信息保全要求：

- 附带 full raw source session ref。
- 关键 fact 尽量带 refs。
- unresolved items 复制到下游。
- `failed` 和 `blocked` 终态包含 failure reason refs。
- `handoff_writer` 可以在定稿前展开 raw refs。
- target Agent 可以通过 context request 展开 raw refs。
- Runtime 记录 handoff 是否基于 incomplete evidence 生成。

高风险任务可以要求 handoff 二次审查：

- release 或 deploy
- security review
- data migration
- permission 或 credential handling
- destructive filesystem changes
- user-facing legal、financial 或 medical content

失败处理

source Agent self-report 缺失时：

- Runtime 使用 structured trace 和 raw refs。
- Handoff status 增加 `self_report_missing` 诊断。

`handoff_writer` 输出未通过校验时：

- Runtime 使用更小、更严格的 Source Bundle 重试。
- 如果重试仍失败，Runtime 从确定性字段创建 minimal handoff。

Minimal handoff：

```
{
  "status": "partial",
  "summary": "Source assignment ended, but handoff writer output was invalid.",
  "artifacts": ["artifact://patch/current"],
  "unresolved": ["Review raw source session before continuing."],
  "raw_refs": ["artifact://raw/session/session_dev_01/full"]
}
```

raw refs 缺失时：

- Runtime 不应将 handoff 标为 fully evidenced。
- target context 应说明 raw evidence missing。
- 高风险 downstream actions 应进入 review 或 approval gate。

Event 类型

Handoff 使用现有 Event 和 Trace 基础设施。

推荐事件类型：

- `handoff.self_report_requested`
- `handoff.source_built`
- `handoff.writer_started`
- `handoff.writer_completed`
- `handoff.writer_failed`
- `handoff.created`
- `handoff.updated`
- `handoff.rejected`
- `handoff.rendered`
- `handoff.consumed`

示例：

```
{
  "id": "evt_handoff_001",
  "seq": 84,
  "time": "2026-06-02T10:30:00Z",
  "scope": "run",
  "type": "handoff.created",
  "actor": {
    "type": "runtime",
    "id": "runtime"
  },
  "run_id": "run_123",
  "assignment_id": "assign_fix_toolbar",
  "data": {
    "handoff_id": "handoff_fix_toolbar_to_review",
    "source_session": "session_dev_01",
    "status": "completed",
    "target_capability": "code_review"
  },
  "refs": [
    "artifact://patch/fix_undo_redo_state",
```

```
"artifact://test/toolbar_passed_001"
],
"prev": "evt_083"
}
```

与现有协议的关系

Model-Runtime Protocol 可以通过 `calls[].handoff` 表达 handoff intent，也可以通过 Runtime call，例如 `handoff.create`，显式创建 handoff。

Runtime 可以提供两个边界调用：

- `handoff.plan`：根据 Assignment 状态、下游依赖、policy、用户触发和预算状态判断是否触发 handoff。
- `handoff.self_report.request`：在 handoff boundary 主动向 source Agent 请求轻量 self-report，并写入事件。

Routing and Delegation Policy 负责 target selection 和 Assignment creation。

State, Event and Projection Model 负责存储 handoff events、handoff chain、trace refs 和 artifact index。

Context, Memory and Visibility Policy 决定 Handoff record 的哪些部分进入 model context、user UI、logs 和 future runs。

UI Console 展示 handoff chain，并允许用户从 session summary、artifact 或 unresolved issue 创建后续 Assignment。

实现路径

第一版实现不应要求普通 Agent prompt 生成完整 Handoff schema。建议路径：

1. 为当前执行边界补充 Runtime trace entries 和 raw refs。
2. 保存 full source session raw ref。
3. terminal Assignment flow 或 Runtime policy 调用 `handoff.plan`。
4. 如果 plan 要求 self-report，调用 `handoff.self_report.request`，向 source Agent 发轻量 plain-text prompt。
5. Runtime 收集 self-report、events、artifacts 和 raw refs，创建 Handoff Source Bundle。
6. 调用 hidden `handoff_writer`。
7. 用扁平 schema 校验 writer output。
8. 保存 canonical Handoff record。
9. 将 Handoff record 渲染成 Markdown，放入 target Context Bundle。

source Agent prompt 可以加入一条轻量规则：

When Runtime asks for a handoff self-report, write concise plain-text notes about completed work, key artifacts, evidence, risks, unresolved questions and suggested next steps. These notes help Runtime build a handoff, but Runtime will validate and rewrite the final handoff record.

这条规则降低格式压力，同时保留 source Agent 的现场判断。

状态、事件、Projection 与 Trace 模型

目的

本文定义 Harness run、Action、Action Graph、Assignment、Artifact、Handoff、Workflow Profile、protocol run、UI projection、audit、evaluation 和 recovery 共享的状态、projection、trace 与 observability 模型。

事实来源

已接受 Event 是规范历史。Projection 是从已接受 Event 派生出来的当前操作视图。

状态文件、数据库行、Action Graph record、Workflow Profile JSON 和场景化 Adapter JSON 可以作为 materialized projection，但不能与已接受 Event 矛盾。

```
Command -> validation -> Event -> Projection -> Trigger/Gate -> next Action/Assignment
```

状态对象分为五类：

- **Event**：Runtime 接受的事实记录。
- **Projection**：从 Event 和 materialized state 推导出的当前操作视图。
- **Materialized State**：用于快速读取、恢复和调度的持久化状态。
- **Trace**：围绕 run、Action、Assignment、Artifact、Gate、Decision 和 Observation 组织出的证据链。
- **Artifact Index**：记录产物 id、来源、状态、可见性、摘要和引用位置。

Trace 与可观测性

Trace 是 Harness run 的结构化可观测视图。它连接已接受 Event、Action record、Assignment record、executor invocation、模型可见 Observation、Artifact refs、gate decision、决策、失败和恢复步骤。

Trace 是协议对象，用于 UI、audit、debugging、evaluation 和 recovery。它不是单独的事实来源，而是从已接受 Event、execution record 以及引用的 artifact/log 派生出来。

Trace record 应同时便于人类 operator 和后续 Agent Session 阅读。Trace entry 应携带稳定 id、status、summary、actor、time、refs、visibility，以及足够的 provenance，用来解释它与当前 Projection 的关系。

Runtime 主动编排也属于 Trace 的证据链。Trace 应能说明 Runtime 为什么触发编排、触发点是什么、来源 Agent Session 是谁、命中了哪个 `orchestration_policy`、生成了哪个 Contract / Handoff Contract、后续 Assignment 获得了什么 authority，以及该编排是否阻塞 parent action 完成。

Trace entry 形态：

```
{
  "id": "trace_action_read_package",
  "run_id": "run_123",
  "kind": "action",
  "status": "completed",
  "summary": "Read package.json and stored a manifest summary.",
  "actor": "runtime",
  "refs": {
    "events": ["evt_041", "evt_042"],
    "action": "action:read_package",
    "artifacts": ["artifact://action/read_package/output"],
    "projection": "projection://run/run_123"
  },
  "visibility": {
    "model": "summary",
    "user": "summary",
    "logs": "full"
  }
}
```

Event Envelope

Event 形态:

```
{
  "id": "evt_123",
  "seq": 42,
  "time": "2026-05-27T12:00:00Z",
  "scope": "run",
  "namespace": "project:open-agent-harness",
  "type": "action.completed",
  "actor": {
    "type": "runtime",
    "id": "runtime"
  },
  "run_id": "run_123",
  "action_id": "read_package",
  "data": {},
  "refs": ["artifact://action/read_package/output"],
  "prev": "evt_122"
}
```

必需属性：

- 稳定 id
- Event log 内单调递增的 sequence
- timestamp
- type
- actor
- scope
- data payload

Orchestration 事件

Runtime 评估和展开 `orchestration_policy` 时，应产生可审计 Event。

事件类型：

- `orchestration_policy.evaluated`
- `orchestration_policy.skipped`
- `orchestration.triggered`
- `orchestration.blocked`
- `orchestration.assignment_created`
- `orchestration.completed`
- `orchestration.failed`
- `handoff.created`

通用执行事件

Action、Assignment、Artifact、Gate 和环境恢复使用共享事件类型：

- `run.created`
- `action.graph_persisted`
- `action.accepted`
- `action.graph_ready`
- `action.started`
- `action.executor_selected`
- `action.permission_requested`
- `action.permission_resolved`
- `action.output_stored`
- `action.completed`
- `action.partially_completed`
- `action.retry_scheduled`
- `action.loop_attempt_started`
- `action.loop_attempt_completed`
- `action.failed`

- `action.cancelled`
- `assignment.created`
- `assignment.started`
- `assignment.completed`
- `assignment.partially_completed`
- `assignment.failed`
- `artifact.declared`
- `artifact.written`
- `artifact.indexed`
- `gate.evaluated`
- `gate.blocked`
- `decision.requested`
- `decision.applied`
- `snapshot.created`
- `rehydration.started`
- `rehydration.completed`
- `rehydration.failed`
- `workflow.created`
- `workflow.saved_from_run`
- `workflow.updated`
- `workflow.run_created`
- `workflow.archived`

`orchestration.triggered` 数据形态:

```
{
  "id": "evt_orchestration_001",
  "seq": 84,
  "time": "2026-05-27T12:08:00Z",
  "scope": "run",
  "namespace": "project:open-agent-harness",
  "type": "orchestration.triggered",
  "actor": {
    "type": "runtime",
    "id": "runtime"
  },
  "run_id": "run_123",
  "assignment_id": "assign_code_developer",
  "orchestration_id": "orchestrate_verify_implementation",
  "data": {
    "policy_id": "verify_implementation",
```

```
    "trigger": "on_completed",
    "source_agent_id": "code_developer",
    "target_capability": "verification",
    "required": true,
    "dedupe_key": "implementation_verification_chain",
    "reason": "Source assignment completed with write side effects and produced
artifact://current/patch."
  },
  "refs": ["artifact://current/patch"],
  "prev": "evt_083"
}
```

Projection 类型

核心 Projection:

- run status
- run goal contract
- action graph
- task/action status
- assignment status
- orchestration status
- orchestration chain
- handoff status
- handoff chain
- open decision queue
- gate status
- artifact index
- trace index
- budget summary
- environment summary
- observability summary
- child session tree
- memory index
- concept state
- UI summary

Projection 应能从 Event 和 stored state 重建。如果重建失败，Runtime 应阻塞 mutation，并暴露 repair decision，而不是在不确定状态下继续执行。

规范状态

不同 Run、Action Graph、Workflow asset 和场景化 Adapter 使用总纲定义的共同状态词汇：

Canonical	含义
draft	已创建，但尚未准备执行。
ready	合法且可调度。
running	正在执行。
waiting_user	等待用户/Owner 输入。
waiting_permission	等待审批。
blocked	缺少决策或前置条件，无法继续。
partial	已产生可用结果，但部分必要子项、验证、handoff、Artifact 或 gate 未完成或未通过。 partial 不等同于 completed；下游只能消费 Runtime 标记为可用的 Artifact，并需要通过 failure、Decision 或 Handoff 处理未决部分。
failed	执行失败。
completed	成功完成。
skipped	被 Decision、Gate 或依赖规则跳过，且跳过行为已记录。
cancelled	Runtime/用户在完成前取消。
aborted	Run 被有意终止为最终状态。

Profile / Adapter 映射：

- Workflow Run completed -> completed
- Workflow Run blocked -> blocked
- Workflow Run waiting_user -> waiting_user
- Workflow Run waiting_permission -> waiting_permission
- Workflow Run failed -> failed
- Workflow Run partial -> partial
- protocol run completed -> completed
- protocol run blocked -> blocked
- protocol run partial -> partial

Artifact Index

Artifact Index 是 Projection 的一部分，记录 run 中可被引用和复用的产物。

Artifact Index 字段：

```
{
  "id": "artifact_patch_current",
  "run_id": "run_123",
  "producer": "action:implement_timeout",
  "type": "patch",
  "name": "current_patch",
  "status": "available",
  "uri": "artifact://run_123/current_patch",
  "summary": "Patch implementing auth timeout handling.",
  "visibility": {
    "model": "summary",
    "user": "summary",
    "logs": "full",
    "future_runs": "ref"
  },
  "evidence": ["event:action.output_stored", "trace:action:implement_timeout"]
}
```

Artifact Index 让 Runtime 在构造模型上下文、UI 视图、Handoff Contract、Trace export 和恢复流程时使用同一个引用。

Materialized State

Materialized State 是状态的持久化形态，用于快速读取、调度、展示和恢复。它包括数据库行、状态文件、Action Graph record、Workflow Profile JSON、场景化 Adapter JSON、artifact index、UI summary cache 和 executor checkpoint。

Materialized State 保存“当前状态”，Projection 保存“当前操作视图”。例如 Action Graph node state 文件可以记录某个 node 的状态、attempt、output 和 error；Projection 会把这些状态和 Event 一起汇总成 run 进度、blocked reason、pending decision 和 UI summary。

Runtime 接受新 mutation 时以 Event 和当前 Projection 为准；Materialized State 提供恢复和查询效率。

事务边界

对任何改变状态的 Command：

1. 校验 schema。

2. 校验 authority。
3. 校验 gate 和当前 Projection。
4. 追加 accepted Event。
5. 更新 Projection。
6. 发出 subscription/update event。

如果 Event append 前一步失败，则不产生状态变更。如果 append 后 Projection update 失败，Runtime 必须将 Projection 标记为 stale，并要求 rebuild，之后才能接受进一步 mutation。

存储层

存储关系：

```
events.jsonl or event table
  -> projections/
  -> traces/
  -> action graph records
  -> workflow profile records
  -> adapter state records
  -> UI/API responses
```

Run、Action Graph、Action、Assignment、Workflow Profile、Workflow Run、Adapter state 和 executor checkpoint 都位于 Harness run store 或受 Runtime 管理的 profile / adapter store 中。它们作为 Materialized State 参与 Projection、Trace、Replay、Export 和恢复。

Replay、Export 与恢复

Replay 从 Event sequence、Materialized State、Artifact Index、Action Graph records、Workflow Profile records 和 Adapter records 重建 Projection。Export 从 Trace、Event、Artifact summaries 和 redaction policy 生成可审计材料。

恢复流程使用以下对象：

- Event Log：确定 Runtime 已接受的事实。
- Projection：判断当前是否可继续执行。
- Materialized State：恢复 run、Action Graph、Action、Assignment、Workflow Profile、Adapter 和 executor 当前状态。
- Snapshot：恢复 workspace、artifact index 或 executor state。
- Manifest：重建 executor 可见环境。
- Trace：解释恢复前后的证据链和决策来源。

Runtime 在恢复后写入 `rehydration.completed` 或 `rehydration.failed` 事件，并更新 Projection 与 Trace。

上下文、Memory 与 Visibility 策略

目的

本文定义 Harness 如何控制 Agent 接收什么信息、存储什么信息、向模型回放什么信息，以及什么信息对用户可见。

核心规则

Runtime 构造上下文。Agent 默认不接收完整原始 transcript。

Projection + Memory Service + Artifact refs -> Context Bundle -> Assignment

Context Bundle 上下文包

Assignment context bundle 应包含：

```
{
  "id": "ctx_123",
  "goal": "Review the changed protocol schema.",
  "included": [
    "file:docs/harness-protocol/02-model-runtime-protocol.md",
    "artifact:protocol_diff"
  ],
  "excluded": [
    "raw transcripts from unrelated child sessions",
    "superseded memory records"
  ],
  "summary": "The protocol supports toolCall carriers and runtime recovery.",
  "memory_refs": ["mem_protocol_policy"],
  "projection_refs": ["runtime://runs/run_123/projections/task-state"]
}
```

Memory 作用域

Memory record 必须声明 scope：

- run
- project
- team

- `global`

Scope precedence 必须明确。当前 Projection 覆盖 historical memory。Project memory 可以在 project scope 内覆盖 team/global preferences。

数据可见性

每个 Action result 都应分类 visibility：

Channel	含义
<code>model</code>	回放到未来模型上下文。
<code>user</code>	展示在 UI 或最终回答中。
<code>logs</code>	为 audit/export 存储。
<code>trace</code>	进入 Trace summary 或 trace export。
<code>future_runs</code>	作为 memory/reference 可供后续 run 使用。
<code>runtime_only</code>	用于控制决策，默认不暴露。

默认策略：

- 模型接收 summary 和 refs
- 用户接收简洁 progress/result projection
- logs 存储完整结构化 trace，并受 redaction 约束
- future runs 接收 curated memory，而不是 raw logs

结果粒度

允许的返回模式：

- `summary`
- `structured`
- `full`
- `on_failure`
- `on_demand`
- `adaptive`

Runtime 可以出于 privacy、safety 或 context budget 原因，将模型可见结果降级到低于请求粒度。Runtime 不能超出策略提升 visibility。

隐私与脱敏

在回放或导出数据前，Runtime 应检查：

- secrets
- credentials
- private keys
- personal data
- proprietary external content
- 包含敏感路径或 token 的 raw command output

Redaction policy 必须一致应用于 tool output、protocol logs、workflow artifacts 和 UI exports。

Shell 与自动 Action

自动 shell execution record 可以在 UI 中可见，同时默认被模型上下文忽略。这与 Harness context isolation 一致：timeline 中可见的输出不会自动成为模型可见上下文。

如果用户明确把 shell output 加入上下文，Runtime 应创建普通 context record，并附带 source refs 和 visibility metadata。

协议能力

Context、Memory 与 Visibility 策略覆盖以下能力：

- 使用 Context Bundle refs 构造模型输入。
- 将 protocol result 以简洁 Observation 形式回放给模型。
- 将完整 output 存储为 Artifact 或 log，并通过 refs 进入上下文。
- 为非模型可见 UI record 提供显式 visibility metadata。
- Memory query result 携带 scope、namespace、status、source、visibility 和 evidence refs。

Workflow Profile 与 Action Graph 资产协议

定位

Harness 使用统一的 Action Graph 表达和执行流程。所有被 Runtime 接受的执行流程都会持久化为 Run、Action Graph、Action、Assignment、Event、Projection、Trace 和 Artifact Index，并支持系统重启后的恢复。

Workflow 是用户创建、保存或命名后的 Action Graph Profile。平时任务执行时由模型生成的图称为 Action Graph；当用户把这个图保存为 Workflow，或用户主动创建 Workflow，它成为 Workflow 资产。

Workflow 和任务执行生成的 Action Graph 使用同一套执行能力：

- 持久化 run state
- dependency DAG
- parallel scheduling
- pause / resume / abort
- retry / failure policy
- gate / approval / decision
- bounded loop
- Agent delegation
- human decision
- Artifact、Trace 和 Projection
- recovery / rehydration
- UI graph projection

差异体现在资产层：

类型	含义
Action Graph	Runtime 当前接受和执行的流程图。
Workflow Profile	可保存、可命名、可复用、可由用户创建的 Action Graph Profile。
Workflow Run	从 Workflow Profile 启动的一次 Run，执行语义仍是 Action Graph。

Action Graph 持久化

每个 `kind: "act"` declaration 被 Runtime 接受后，都会创建或更新持久化 Run，并写入 Action Graph record。

Runtime 持久化以下对象：

- Run record
- Action Graph record
- Action records
- dependency edges
- Assignment records
- Artifact Index
- Event Log
- Projection
- Trace
- Decision records
- Snapshot / Manifest refs

短任务可以很快完成，但它仍然是可审计、可回放、可恢复的持久化 Run。系统重启后，Runtime 根据 Event、Projection、Materialized State、Artifact Index、Snapshot 和 Manifest 判断哪些 Action 已完成、哪些仍在运行、哪些需要恢复、哪些应转为 blocked 或 waiting 状态。

Action Graph 能力

Action Graph 是执行流程的统一语义对象。

顶层能力：

能力	说明
goal / criteria	Run 或 graph 的目标和完成标准。
inputs	用户输入、参数、Artifact refs 或环境 refs。
variables	Runtime 在执行中维护的结构化变量。
calls[] / nodes[]	可调度 Action 节点。模型侧通常使用 calls[]。
depends_on	节点依赖边。
failure	失败处理策略，例如 retry、block、ask_user、handoff、abort。
gate	approval、verification、review、privacy、release 等状态迁移条件。
loop	有边界的重复执行语义。
budget	cost、timeout、attempt、parallelism、token 和资源预算。
visibility	输出进入 model、user、logs、trace、future_runs 或 runtime_only 的规则。

能力	说明
artifacts	期望产生、读取、更新或引用的 Artifact。
handoff	跨 executor 或 Agent Session 的结构化交接。

这些能力可以来自三个来源：

- 模型在 Action Graph 中显式声明。
- Runtime 根据系统策略和当前 Projection 补齐。
- Agent metadata / Orchestration Policy 为特定 Agent Session 提供默认策略。

例如 retry、loop、verification、handoff 和 post-action review 可以由 Action Graph 字段表达，也可以由 Agent 模板上的 orchestration policy 触发。Runtime 在接受和执行前将这些来源合并为统一的 Action、Assignment 和 Gate。

Profile 形态

模型侧保持扁平 `calls[]`。Workflow Profile 可以使用更适合 UI 和用户编辑的字段，但语义仍映射到 Action Graph。

示例：

```
{
  "id": "wf_auth_timeout",
  "title": "Fix auth timeout handling",
  "goal": {
    "summary": "Implement typed timeout error handling for auth refresh.",
    "constraints": ["keep_public_api", "touch_auth_module_only"],
    "criteria": [
      "Typed timeout error is returned by the refresh path.",
      "Focused tests cover success and timeout cases."
    ]
  },
  "budget": {
    "cost": "medium",
    "timeout_ms": 1800000,
    "max_parallel": 2
  },
  "failure": {
    "on_node_failed": "ask_decision",
    "retry": { "max_attempts": 2 }
  },
  "nodes": [
    {
```

```

    "id": "inspect",
    "type": "agent",
    "name": "explore",
    "title": "Inspect auth refresh flow",
    "args": {
      "prompt": "Find the auth refresh timeout path and relevant tests."
    },
    "criteria": [
      "Relevant source files and tests are identified.",
      "The current timeout behavior is summarized with evidence."
    ],
    "artifacts": [
      { "name": "auth_research", "type": "research_note" }
    ]
  },
  {
    "id": "implement",
    "type": "agent",
    "name": "backend",
    "title": "Implement timeout handling",
    "args": {
      "prompt": "Apply the code change using the inspect result."
    },
    "depends_on": ["inspect"],
    "mutates": true,
    "criteria": [
      "Code implements typed timeout error behavior.",
      "Changed files are listed in the result."
    ],
    "artifacts": [
      { "name": "patch", "type": "diff" }
    ]
  },
  {
    "id": "verify",
    "type": "agent",
    "name": "verifier",
    "title": "Verify timeout handling",
    "args": {
      "prompt": "Run focused tests and typecheck for the changed behavior."
    },
    "depends_on": ["implement"],

```

```
"criteria": [
  "Focused tests pass.",
  "Relevant package typecheck passes."
],
"gate": {
  "required": true
},
"artifacts": [
  { "name": "test_report", "type": "verification" }
]
},
{
  "id": "review",
  "type": "agent",
  "name": "technical-reviewer",
  "title": "Review implementation",
  "args": {
    "prompt": "Review the patch for correctness, regressions and test coverage."
  },
  "depends_on": ["verify"],
  "criteria": [
    "Findings include severity and evidence.",
    "Review decision is approve, request_rework or needs_info."
  ],
  "artifacts": [
    { "name": "review_report", "type": "review" }
  ]
}
]
```

Runtime 可以把 `nodes[]` 归一化为模型侧等价的 `calls[]`，再进入统一 Action Graph 执行路径。

Workflow 资产

Workflow 资产记录一个可复用 Action Graph Profile 及其治理信息。

Workflow asset 字段：

字段	含义
<code>id</code>	Workflow 稳定 id。
<code>title</code>	用户可读标题。

字段	含义
<code>description</code>	简要说明。
<code>profile</code>	Action Graph Profile。
<code>inputs_schema</code>	用户启动时需要填写的输入。
<code>owner</code>	创建者或治理 owner。
<code>version</code>	Profile 版本。
<code>source</code>	<code>user_created</code> 、 <code>saved_from_run</code> 、 <code>imported</code> 或 <code>generated</code> 。
<code>visibility</code>	谁可以看到、运行、编辑或复用。
<code>created_at</code> / <code>updated_at</code>	时间戳。

Workflow 的创建方式：

- 用户从 UI 创建 Workflow。
- 用户把一次 Action Graph 保存为 Workflow。
- Agent 生成 Action Graph 后，用户确认保存为 Workflow。
- 外部系统导入 Workflow Profile。

保存为 Workflow 不改变执行语义。它只让这份 Action Graph Profile 获得名称、版本、输入 schema、权限和 UI 管理入口。

Loop 与 Retry

Action Graph 使用有边界策略表达重复执行。

Loop 示例：

```
{
  "id": "stabilize",
  "type": "loop",
  "title": "Stabilize implementation",
  "depends_on": ["implement"],
  "loop": {
    "max_attempts": 3,
    "until": [
      { "type": "variable", "name": "stabilize.test", "equals": "passed" }
    ],
    "steps": [
      {
        "id": "test",
```

```

    "type": "agent",
    "name": "verifier",
    "args": {
      "prompt": "Run focused verification. Return `passed` or failure details."
    },
    "outputs": { "stabilize.test": "$result.status" }
  },
  {
    "id": "fix",
    "type": "agent",
    "name": "debugger",
    "mutates": true,
    "args": {
      "prompt": "Fix the reported failure and summarize changed files."
    },
    "context": {
      "include": ["steps.test.output", "attempts.previous.fix", "artifacts.patch"]
    }
  }
]
}
}

```

Runtime 语义:

- `max_attempts` 是硬边界。
- `until` 根据 Run variables、Action output 或 Artifact summary 判断。
- 每个 loop step 归一化为 Action。
- loop attempt、child output、Artifact、failure 和 Decision 都写入 Trace。
- loop 可以由 Action Graph 显式声明，也可以由 Agent metadata 的 orchestration policy 触发。

Retry 示例:

```

{
  "id": "run_tests",
  "type": "agent",
  "name": "verifier",
  "failure": {
    "retry": {
      "max_attempts": 2,
      "when": ["transient_failure", "tool_timeout"]
    },
    "on_exhausted": "block"
  }
}

```

```
}  
}
```

Gate、Decision 与 Handoff

Gate、Decision 和 Handoff 也是 Action Graph 能力。

常见模式：

```
implement -> verify -> review -> gate -> summarize
```

含义：

- `verify` 产生可引用验证证据。
- `review` 评估正确性、回归风险和测试覆盖。
- `gate` 聚合 verification、approval、budget、privacy 或 release 条件。
- `decision` 处理 blocked、retry、skip、abort、request_input 或 replan。
- `handoff` 将结果、证据、Artifact、风险和未决问题交给后续 executor。

Gate 可以由 deterministic tool、Runtime service、Agent Session 或 human executor 完成。

Workflow Command

Workflow 相关 UI/API 操作通过 Command 进入 Runtime。

Command 类型：

- `workflow.create`：创建 Workflow asset。
- `workflow.save_from_run`：将某次 Run 的 Action Graph Profile 保存为 Workflow asset。
- `workflow.update`：更新 Workflow asset 的 profile、输入 schema、标题、说明或可见性。
- `workflow.run`：从 Workflow asset 创建新的 Run，并 materialize 出新的 Action Graph。
- `workflow.archive`：归档 Workflow asset。
- `workflow.delete`：删除或软删除 Workflow asset，按权限和审计策略执行。

这些 Command 都写入 Event、Projection 和 Trace。`workflow.run` 产生的新 Run 与任务执行生成的 Action Graph Run 使用同一套状态机。

执行语义

Runtime 执行 Action Graph 时遵循以下语义：

1. 接受模型 declaration、UI Command 或 Workflow Profile。
2. 创建或更新持久化 Run。
3. 写入 Action Graph record、Action records 和 dependency edges。

4. 校验 duplicate id、missing dependency、cycle、authority、budget、gate、Artifact Contract 和 side effect。
5. 计算 ready Actions，并按 policy 调度。
6. 需要 Agent 执行时创建 Assignment 和 Agent Session。
7. Executor 结果写入 Action result、Artifact、Event 和 Trace。
8. Runtime 根据 result、criteria、gate、failure 和 orchestration policy 更新状态。
9. 阻塞、审批、失败和 replan 进入 Decision 或 Handoff 流程。
10. 所有 required Actions 和 gates 满足后，Run 进入 `completed`。

并行度由 Runtime 依据 ready Actions、resource lock、budget、provider/model limit、workspace safety 和 UI policy 决定。

Recovery 与 Rehydration

Action Graph 恢复使用统一状态模型：

- Event Log 确定 Runtime 接受过的事实。
- Projection 判断 Run 当前状态和可调度 Action。
- Materialized State 提供 Run、Action、Assignment、Decision、Workflow asset 和 Artifact 当前状态。
- Snapshot 记录 workspace、artifact index 或 executor checkpoint。
- Manifest 重建 executor 可见环境。
- Trace 解释恢复前后的证据链。

恢复流程：

1. 加载 Run record、Action Graph、Action records、Assignment records 和 Artifact Index。
2. 从 Event sequence 重建 Projection。
3. 检查 running Assignment 对应的 Agent Session 或 executor invocation。
4. 将可恢复 executor 通过 Manifest 和 Snapshot rehydrate。
5. 将缺少 owner 的 running Action 投影为 `blocked`，并创建 Decision request。
6. 重新计算 ready Actions。
7. 继续调度或等待用户、权限、Decision。

UI 与 Agent 可读产物

Action Graph Projection、Workflow asset、Trace、Artifact summary 和 Action records 面向人类和 Agent Session 可读。

UI 展示：

- Run title、goal、status 和 progress
- Action Graph 和依赖关系
- running / blocked / failed Actions

- pending Decision 和 approval
- Artifact Index
- child Agent Session trace
- budget 和 gate 状态
- recovery / replay / export 入口
- Workflow asset 列表、版本、输入 schema 和运行历史

Agent Session 接收：

- Action / Assignment task
- Contract
- upstream Artifact refs
- criteria
- failure
- budget
- visibility
- current Projection summary
- relevant Trace summary

Runtime 通过结构化 refs 连接 UI、Trace、Artifact 和 Agent Context，避免系统产物只适合人读。

UI 控制台与 Agent 管理协议

目的

Harness UI 是用户管理、观察和使用 Harness 系统的操作面。UI 读取 Runtime Projection、Trace、Event、Artifact、Memory 和 Agent registry；用户操作通过 Command、受控 Action 或场景化 operation 进入 Runtime。

UI 不从自然语言 transcript 推断系统状态。它展示 Runtime 已接受和投影出的结构化状态，并让用户在合适边界内创建 run、切换 Agent、进入不同 Agent Session、批准决策、恢复执行、审查证据和导出 trace。

Surface

UI 由五类 surface 组成：

- **Harness Console**：管理 Run、Task、Action、Assignment、Gate、Decision、Artifact、Event、Projection 和 Trace。
- **Agent Session Workbench**：展示 root session、child session、descendant session，并允许用户进入不同 Agent Session 继续交互。
- **Agent Manager**：管理 Agent 模板、entry、capability、permission、model preference、relationships、orchestration policy 和启用状态。
- **Protocol / Workflow Panel**：观察模型 DSL、Action Graph、Workflow 资产、executor routing、state record 和恢复状态。
- **Governance View**：观察 Memory、Concept、Authority、Gate、Trace export、redaction 和 audit evidence。

数据读取模型

UI 默认读取 Projection。Event Log、raw trace、executor logs 和 Artifact 原文作为证据入口存在，不作为常规状态源。

UI 读取对象：

- Run
- Task
- Action
- Assignment
- Agent Session
- Artifact
- Gate
- Decision
- Event

- `Trace`
- `Memory`
- `Concept`
- `Agent`
- `Workflow Profile`
- `Workflow Asset`
- `Workflow Run`

每个 UI record 都应包含稳定 id、status、summary、refs、evidence、actor、time、visibility 和 provenance，让人类和后续 Agent Session 都能读取和复用。

Command 模型

会改变系统状态的 UI 操作提交为 Command。Runtime 校验 schema、authority、gate、当前 Projection 和 redaction policy 后，追加 Event 并更新 Projection。

Command 类型：

- `run.create`
- `run.pause`
- `run.resume`
- `run.abort`
- `task.retry`
- `task.cancel`
- `action.retry`
- `assignment.cancel`
- `decision.answer`
- `permission.approve`
- `permission.reject`
- `verify.rerun`
- `session.message.submit`
- `session.focus`
- `workflow.create`
- `workflow.save_from_run`
- `workflow.update`
- `workflow.run`
- `workflow.archive`
- `agent.enable`
- `agent.disable`
- `agent.update`
- `concept.replace.request`

- `projection.rebuild.request`
- `trace.export.request`

Command result 返回更新后的 Projection summary、Event refs、Trace refs 和可见的 blocker / rejection reason。失败的 Command 不产生部分状态。

Harness Console

Harness Console 展示受治理 run 的当前状态和证据链。

核心视图：

- Run list: status、goal、owner、current phase、progress、blocked reason、pending decision、updated time。
- Run detail: Goal Contract、Action Graph、Assignment list、Artifact Index、Gate state、Decision queue、Trace summary。
- Task / Action detail: operation、executor、depends_on、criteria、failure、budget、visibility、status、result、artifacts、events。
- Assignment detail: Agent Session、authority、Context Bundle summary、contract、result、unresolved、child trace。
- Gate detail: gate 输入、判定结果、证据、阻塞原因、可用 Decision。
- Event Explorer: 按 sequence、type、actor、scope、refs 和 status 检查已接受事实。
- Audit Export: 按 visibility 和 redaction policy 导出 trace、event、artifact summary 和 decision evidence。

Console 中的 graph、timeline、progress、summary 和 comparison view 都是 Projection 或 Trace 的展示形态。

多 Agent Session 交互

用户可以与不同 Agent Session 交互，而不是只能停留在一个主会话里。

Session Workbench 展示：

- root session
- child session
- descendant session
- Workflow asset management session
- review / test / debug / research 等 specialist session
- waiting_user、waiting_permission、blocked、partial、failed 等状态标记

用户可以：

- 打开任意 Agent Session 查看 session log、Context Bundle 摘要、Assignment、authority、Artifact refs 和 Trace。

- 在某个 Agent Session 内提交补充输入，形成 `session.message.submit` Command。
- 对 `waiting_user` / `waiting_permission` 的 session 提供回答、批准或拒绝。
- 从 parent session 跳转到 child session，也可以从 child session 回到 parent run projection。
- 查看某个 Agent Session 的上下文来源，包括 Projection、Memory、Artifact、Trace 和语义解释提示。
- 将某个 session 的 Artifact、summary 或 unresolved issues 作为 Handoff 输入交给后续 Agent Session。

Agent Session 之间仍不直接通信。用户在 UI 中进入某个 session，是把输入提交给 Runtime；Runtime 再根据该 session 的 authority、Assignment、Context Bundle 和当前 Projection 构造下一次模型调用。

Agent Manager

Agent Manager 把 Agent 作为受治理 runtime object 管理。

它展示：

- Agent id、name、description、persona
- source: package、user、project
- entry flags: primary、delegable、mentionable、default、hidden
- capability: purpose、tags、cost、writes
- permission: permission_mode、allowed_tools、denied_tools、inherit_permissions
- model preference
- relationships
- orchestration_policy
- enabled / disabled state
- diagnostics

用户可以创建、编辑、启用、禁用 user/project Agent 模板。Package / builtin 模板作为只读来源展示。Disabled Agent 仍在 Agent Manager 中可见，但不会进入普通 picker、mention suggestions 或 delegation candidate。

Agent Manager 需要区分 capability 和 authority：capability 说明 Agent 适合做什么，authority 由 Runtime 在具体 Assignment 中授予。

Protocol 与 Workflow Panel

Protocol Panel 展示模型与 Runtime 的结构化交互。

它展示：

- model declaration
- recovered tool request
- Action Graph
- selected Action detail
- executor routing

- result
- Runtime Observation
- protocol error
- recovery event
- trace export

Workflow Panel 展示用户创建、保存或命名后的 Workflow 资产，以及从这些资产启动的 Workflow Run。它展示：

- Workflow Profile
- Workflow asset metadata
- Workflow Run history
- materialized Action Graph
- node status
- ready / running / blocked / partial / failed nodes
- loop attempt
- Decision queue
- Handoff chain
- Artifact Index
- recovery / rehydration status
- workflow trace

Protocol Panel 和 Workflow Panel 使用同一套字段、状态词、Artifact refs、Trace refs 和 visibility。Workflow Run 的执行能力与任务执行生成的 Action Graph 相同；差异在于 Workflow 是可保存、可命名、可复用和可管理的资产。

Governance View

Governance View 面向权限、记忆、概念和审计。

它展示：

- Authority View：某个 Run、Action、Assignment 或 Agent Session 的生效 authority。
- Memory View：run、project、team、global scope 下的 Memory refs、status、freshness、evidence 和 visibility。
- Concept View：active、superseded、historical concept，以及 replacement request、impact refs 和 gate result。
- Trace View：Command -> Event -> Projection -> Action -> Executor -> Artifact -> Observation 的证据链。
- Redaction View：导出或回放前应用的脱敏规则和被隐藏字段。

Concept replacement、projection rebuild、trace export 和高影响 redaction override 都通过 Command 进入 Runtime，并按决策边界升级。

Agent 可读 UI 产物

UI 产物需要同时服务人和 Agent Session。

因此，UI 中的摘要、表格、graph node、timeline entry、decision card、artifact card 和 trace export 都应提供：

- 稳定 id
- status
- summary
- refs
- evidence
- actor
- time
- visibility
- source projection
- unresolved issues

后续 Agent Session 可以通过 Context Bundle 引用这些 UI 产物，而不是重新读取完整 transcript 或 raw logs。

Visibility 与 Redaction

UI 遵守 `visibility` 字段。

- `model` 控制是否进入后续模型上下文。
- `user` 控制是否展示给用户。
- `logs` 控制是否进入审计日志。
- `trace` 控制是否进入 trace export。
- `future_runs` 控制是否成为后续 run 可引用材料。
- `runtime_only` 控制是否只用于 Runtime 内部决策。

Trace export、Artifact preview、raw log view、Memory promotion 和 Agent Session context preview 都需要应用 redaction policy。包含 secret、credential、private key、personal data、敏感路径或未授权外部内容的材料，只能以摘要、引用或脱敏形式展示和复用。

UI 协议能力

UI 管理协议覆盖以下能力：

- 通过 Projection 观察 Harness 状态。
- 通过 Command 推进 Run、Task、Action、Assignment、Decision、Agent 和 Concept。
- 与 root、child、descendant Agent Session 分别交互。
- 管理 Agent 模板和启用状态。

- 观察模型 DSL、Action Graph、Workflow 资产、executor routing 和 Runtime Observation。
- 检查 authority、gate、memory、concept、artifact、trace 和 audit evidence。
- 将 UI 产物作为 agent-readable refs 供后续 Context Bundle 使用。